INFORME FASE 4

En esta fase probamos cinco índices para comprobar si mejoraban el rendimiento de algunas consultas. Comparamos los planes de ejecución antes y después, analizamos los índices que no resultaron útiles y medimos cuánto costó mantener índices adicionales al insertar registros. Por último, verificamos que los cinco índices funcionaran juntos.

1. Objetivo y Alcance

El objetivo fue elegir consultas del proyecto, medir cómo funcionaban y decidir qué índices convenía agregar. Nos guiamos por la Etapa 4 de la consigna (páginas 14 y 15) y por las consultas de la sección 8.

Utilizamos la base cargada en la Fase 3, con 27 tablas que contienen datos. Para estas pruebas resultaron importantes comprobante (150.000 filas), renglon_comprobante (500.000), cliente (20.000), transferencia (2.000) y renglon_transferencia (8.000). Cada integrante trabajó sobre una copia de prueba, sin modificar la base original ni el respaldo.

2. Entorno y metodología de evaluación

Al comenzar había 33 índices generados por las claves primarias y las restricciones UNIQUE. Las pruebas se realizaron con PostgreSQL 18, en distintas versiones según el equipo (18.1, 18.3 y 18.6). Por eso comparamos los tiempos antes y después dentro de cada prueba, sin comparar directamente la velocidad de las computadoras de los integrantes.

Primero actualizamos las estadísticas con ANALYZE; en las pruebas de Antonio también se utilizó VACUUM ANALYZE. Después ejecutamos EXPLAIN (ANALYZE, BUFFERS) antes y después de crear cada índice. Hicimos tres mediciones por escenario y usamos la mediana del tiempo de ejecución. No obligamos a PostgreSQL a usar índices: comprobamos en cada plan si realmente los elegía.

Calculamos la mejora como (tiempo antes − tiempo después) / tiempo antes × 100. Para considerar que la mejora se debía al índice, también revisamos que el plan lo utilizara, y no solo que el tiempo fuera menor.

Un detalle importante de los datos es que todos los comprobantes comerciales tienen fecha 30/10/2026. Por eso algunos filtros de fechas, como “último trimestre”, no ayudan a reducir la cantidad de registros que se deben consultar.

3. Diseño e integración de índices

Índice definitivo	Tabla y definición principal	Justificación
idx_comprobante_sala_fecha	comprobante (id_sala, fecha)	Compuesto: igualdad por sala y rango temporal en S3.
idx_rc_cod_vino	renglon_comprobante (cod_vino)	FK: acceso selectivo por vino; evaluado con vino 442.
idx_comprobante_id_cliente_inc	comprobante (id_cliente) INCLUDE (nro_comprobante)	Especial / FK: acceso cubierto para S12-A.
idx_rt_cod_vino_nro	renglon_transferencia (cod_vino, nro_solicitud)	Compuesto: buscar vino y relacionar transferencia en S4.
idx_transferencia_pendiente_sala	transferencia (id_sala) WHERE estado <> ‘procesado’	Parcial: filas aún no procesadas, S5; ausencia de casos positivos en dump.

En el índice de S3 colocamos primero id_sala, porque la consulta busca una sala específica, y después fecha, porque filtra por un período. En S4 usamos (cod_vino, nro_solicitud): no es igual a la clave primaria existente (nro_solicitud, cod_vino), ya que la consulta comienza buscando por vino. Los índices que no aportaron mejoras no se incluyeron en la migración definitiva.

Los cinco índices quedaron definidos en db/migrations/008_indices.sql. Al aplicarlos en la base de integración, la consulta al catálogo pg_indexes confirmó 38 índices en total: los 33 anteriores y los cinco nuevos.

4. Comparación de consultas oficiales

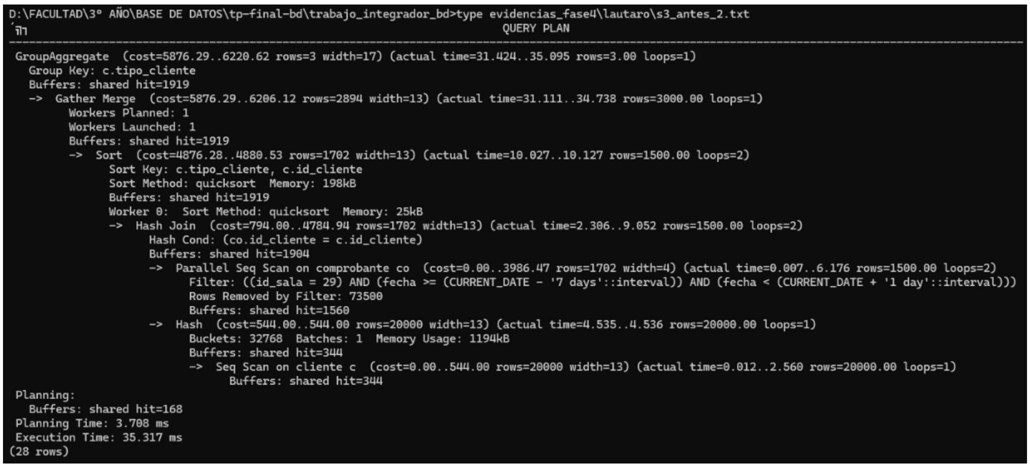
Consulta	Antes (ms)	Después (ms)	Mejora	Recorrido	Filas revisadas / descartadas
S3 – Sala 29	35,317	12,623	64,26%	Parallel Seq Scan → Bitmap Index Scan	150.000 revisadas; 147.000 descartadas → 3.000 encontradas
S4 – Vino 681	1,041	0,719	30,93%	Seq Scan → Bitmap Index Scan	8.000 revisadas; 7.980 descartadas → 20 encontradas
S12-A – Bodega DEMO Atuel 001	104.823,719	365,214	99,65%	Seq Scan reiterado → Index Only Scan	20.370 recorridos; ~148.592 descartadas por recorrido → búsquedas por cliente

La tabla muestra las medianas de las pruebas individuales. También indicamos cómo cambió el acceso a las tablas y cuántas filas se revisaron o descartaron, según lo que informa cada plan. Más adelante probamos los cinco índices juntos para verificar que siguieran utilizándose.

4.1. S3 – Clientes por tipo en una sala durante la última semana

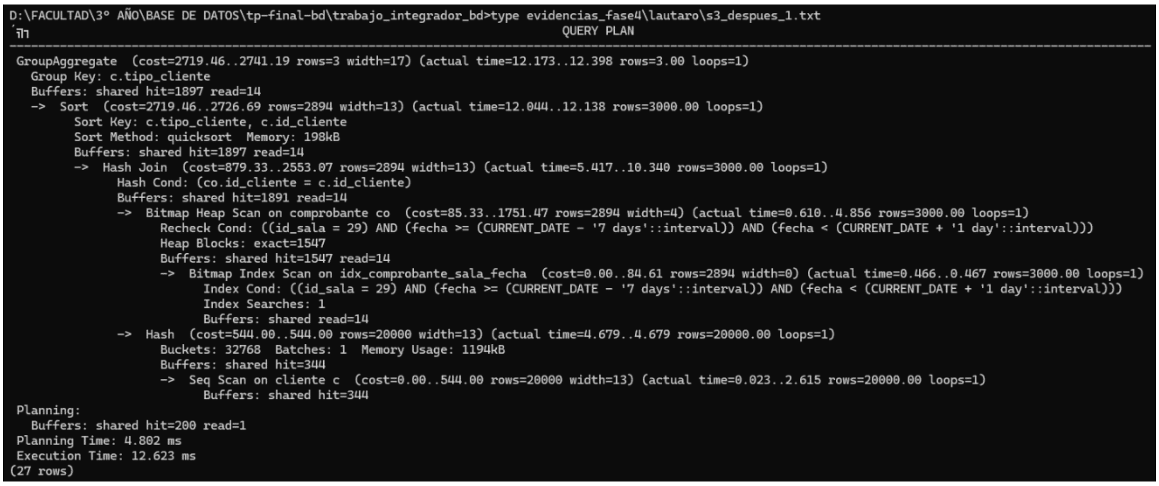
Para la sala 29, sin el índice PostgreSQL recorría los 150.000 comprobantes y descartaba 147.000 para encontrar los 3.000 que cumplían el filtro. Después de crear idx_comprobante_sala_fecha, pasó a buscar primero en el índice y luego a consultar los bloques de tabla necesarios. La unión con cliente siguió siendo la misma.

El tiempo bajó de 35,317 ms a 12,623 ms: una mejora del 64,26 %. En esta base, la diferencia se explica principalmente por la selección de la sala, porque todos los comprobantes tienen la misma fecha.



```
D:\FACULTAD\3° AÑO\BASE DE DATOS\tp-final-bd\trabajo_integrador_bd>type evidencias_fase4\lautaro\s3_antes_2.txt
´╗┐                                                                                                  QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------------------------------------
 GroupAggregate  (cost=5876.29..6220.62 rows=3 width=17) (actual time=31.424..35.095 rows=3.00 loops=1)
   Group Key: c.tipo_cliente
   Buffers: shared hit=1919
   ->  Gather Merge  (cost=5876.29..6206.12 rows=2894 width=13) (actual time=31.111..34.738 rows=3000.00 loops=1)
         Workers Planned: 1
         Workers Launched: 1
         Buffers: shared hit=1919
         ->  Sort  (cost=4876.28..4880.53 rows=1702 width=13) (actual time=10.027..10.127 rows=1500.00 loops=2)
               Sort Key: c.tipo_cliente, c.id_cliente
               Sort Method: quicksort  Memory: 198kB
               Buffers: shared hit=1919
               Worker 0:  Sort Method: quicksort  Memory: 25kB
               ->  Hash Join  (cost=794.00..4784.94 rows=1702 width=13) (actual time=2.306..9.052 rows=1500.00 loops=2)
                     Hash Cond: (co.id_cliente = c.id_cliente)
                     Buffers: shared hit=1904
                     ->  Parallel Seq Scan on comprobante co  (cost=0.00..3986.47 rows=1702 width=4) (actual time=0.007..6.176 rows=1500.00 loops=2)
                           Filter: ((id_sala = 29) AND (fecha >= (CURRENT_DATE - '7 days'::interval)) AND (fecha < (CURRENT_DATE + '1 day'::interval)))
                           Rows Removed by Filter: 73500
                           Buffers: shared hit=1560
                     ->  Hash  (cost=544.00..544.00 rows=20000 width=13) (actual time=4.535..4.536 rows=20000.00 loops=1)
                           Buckets: 32768  Batches: 1  Memory Usage: 1194kB
                           Buffers: shared hit=344
                           ->  Seq Scan on cliente c  (cost=0.00..544.00 rows=20000 width=13) (actual time=0.012..2.560 rows=20000.00 loops=1)
                                 Buffers: shared hit=344
 Planning:
   Buffers: shared hit=168
 Planning Time: 3.708 ms
 Execution Time: 35.317 ms
(28 rows)
```

S3 antes: recorrido secuencial paralelo sobre comprobante.



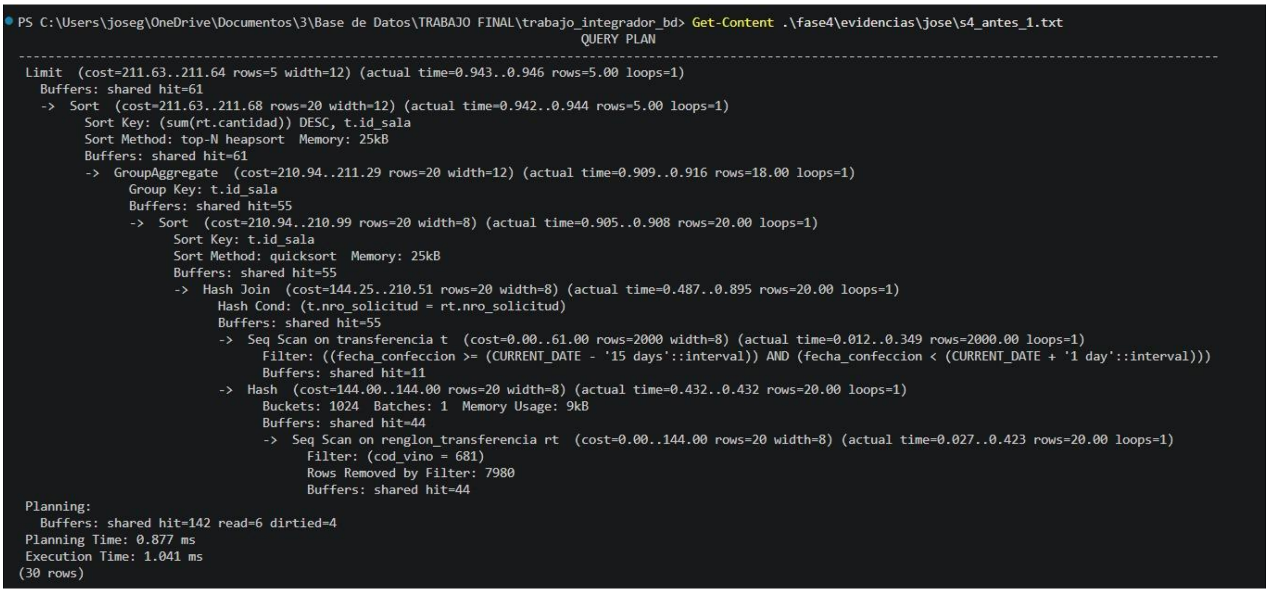
```
D:\FACULTAD\3º AÑO\BASE DE DATOS\tp-final-bd\trabajo_integrador_bd>type evidencias_fase4\lautaro\s3_despues_1.txt
´ī7                                                                 QUERY PLAN
----------------------------------------------------------------------------------------------------------------------------------------------------------------
 GroupAggregate  (cost=2719.46..2741.19 rows=3 width=17) (actual time=12.173..12.398 rows=3.00 loops=1)
   Group Key: c.tipo_cliente
   Buffers: shared hit=1897 read=14
   ->  Sort  (cost=2719.46..2726.69 rows=2894 width=13) (actual time=12.044..12.138 rows=3000.00 loops=1)
         Sort Key: c.tipo_cliente, c.id_cliente
         Sort Method: quicksort  Memory: 198kB
         Buffers: shared hit=1897 read=14
         ->  Hash Join  (cost=879.33..2553.07 rows=2894 width=13) (actual time=5.417..10.340 rows=3000.00 loops=1)
               Hash Cond: (co.id_cliente = c.id_cliente)
               Buffers: shared hit=1891 read=14
               ->  Bitmap Heap Scan on comprobante co  (cost=85.33..1751.47 rows=2894 width=4) (actual time=0.610..4.856 rows=3000.00 loops=1)
                     Recheck Cond: ((id_sala = 29) AND (fecha >= (CURRENT_DATE - '7 days'::interval)) AND (fecha < (CURRENT_DATE + '1 day'::interval)))
                     Heap Blocks: exact=1547
                     Buffers: shared hit=1547 read=14
                     ->  Bitmap Index Scan on idx_comprobante_sala_fecha  (cost=0.00..84.61 rows=2894 width=0) (actual time=0.466..0.467 rows=3000.00 loops=1)
                           Index Cond: ((id_sala = 29) AND (fecha >= (CURRENT_DATE - '7 days'::interval)) AND (fecha < (CURRENT_DATE + '1 day'::interval)))
                           Index Searches: 1
                           Buffers: shared read=14
               ->  Hash  (cost=544.00..544.00 rows=20000 width=13) (actual time=4.679..4.679 rows=20000.00 loops=1)
                     Buckets: 32768  Batches: 1  Memory Usage: 1194kB
                     Buffers: shared hit=344
                     ->  Seq Scan on cliente c  (cost=0.00..544.00 rows=20000 width=13) (actual time=0.023..2.615 rows=20000.00 loops=1)
                           Buffers: shared hit=344
 Planning:
   Buffers: shared hit=200 read=1
 Planning Time: 4.802 ms
 Execution Time: 12.623 ms
(27 rows)
```

S3 después: Bitmap Index Scan usando el índice compuesto.

4.2. S4 – Salas que solicitaron mas unidades de un vino

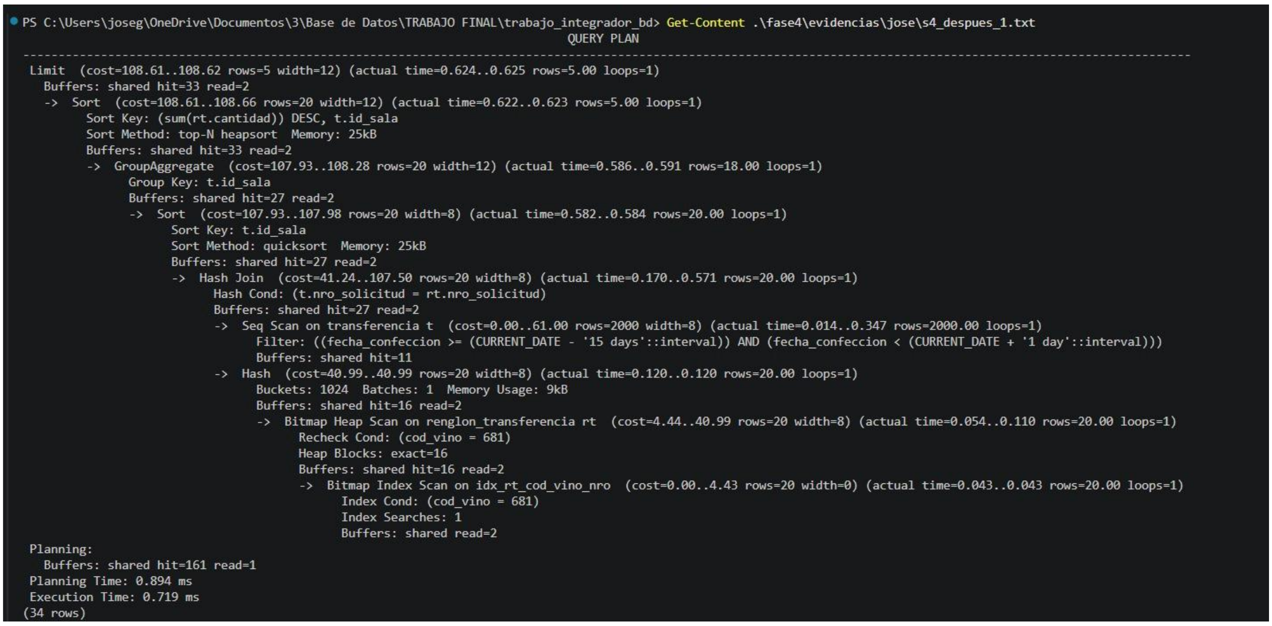
Para el vino 681 encontramos 20 renglones de transferencia, correspondientes a 20 solicitudes de 18 salas. Sin índice, PostgreSQL revisaba los 8.000 renglones y descartaba 7.980. Con idx_rt_cod_vino_nro pasó a localizar los 20 renglones mediante el índice.

El tiempo bajó de 1,041 ms a 0,719 ms, una mejora del 30,93 %. Aunque la diferencia es de solo 0,322 ms, el plan muestra que se evitaron miles de comprobaciones innecesarias.



```
PS C:\Users\joseg\OneDrive\Documentos\3\Base de Datos\TRABAJO FINAL\trabajo_integrador_bd> Get-Content .\fase4\evidencias\jose\s4_antes_1.txt
                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=211.63..211.64 rows=5 width=12) (actual time=0.943..0.946 rows=5.00 loops=1)
   Buffers: shared hit=61
   ->  Sort  (cost=211.63..211.68 rows=20 width=12) (actual time=0.942..0.944 rows=5.00 loops=1)
         Sort Key: (sum(rt.cantidad)) DESC, t.id_sala
         Sort Method: top-N heapsort  Memory: 25kB
         Buffers: shared hit=61
         ->  GroupAggregate  (cost=210.94..211.29 rows=20 width=12) (actual time=0.909..0.916 rows=18.00 loops=1)
               Group Key: t.id_sala
               Buffers: shared hit=55
               ->  Sort  (cost=210.94..210.99 rows=20 width=8) (actual time=0.905..0.908 rows=20.00 loops=1)
                     Sort Key: t.id_sala
                     Sort Method: quicksort  Memory: 25kB
                     Buffers: shared hit=55
                     ->  Hash Join  (cost=144.25..210.51 rows=20 width=8) (actual time=0.487..0.895 rows=20.00 loops=1)
                           Hash Cond: (t.nro_solicitud = rt.nro_solicitud)
                           Buffers: shared hit=55
                           ->  Seq Scan on transferencia t  (cost=0.00..61.00 rows=2000 width=8) (actual time=0.012..0.349 rows=2000.00 loops=1)
                                 Filter: ((fecha_confeccion >= (CURRENT_DATE - '15 days'::interval)) AND (fecha_confeccion < (CURRENT_DATE + '1 day'::interval)))
                                 Buffers: shared hit=11
                           ->  Hash  (cost=144.00..144.00 rows=20 width=8) (actual time=0.432..0.432 rows=20.00 loops=1)
                                 Buckets: 1024  Batches: 1  Memory Usage: 9kB
                                 Buffers: shared hit=44
                                 ->  Seq Scan on renglon_transferencia rt  (cost=0.00..144.00 rows=20 width=8) (actual time=0.027..0.423 rows=20.00 loops=1)
                                       Filter: (cod_vino = 681)
                                       Rows Removed by Filter: 7980
                                       Buffers: shared hit=44
 Planning:
   Buffers: shared hit=142 read=6 dirtied=4
 Planning Time: 0.877 ms
 Execution Time: 1.041 ms
(30 rows)
```

S4 antes: Seq Scan sobre 8.000 renglones, con 7.980 descartados.



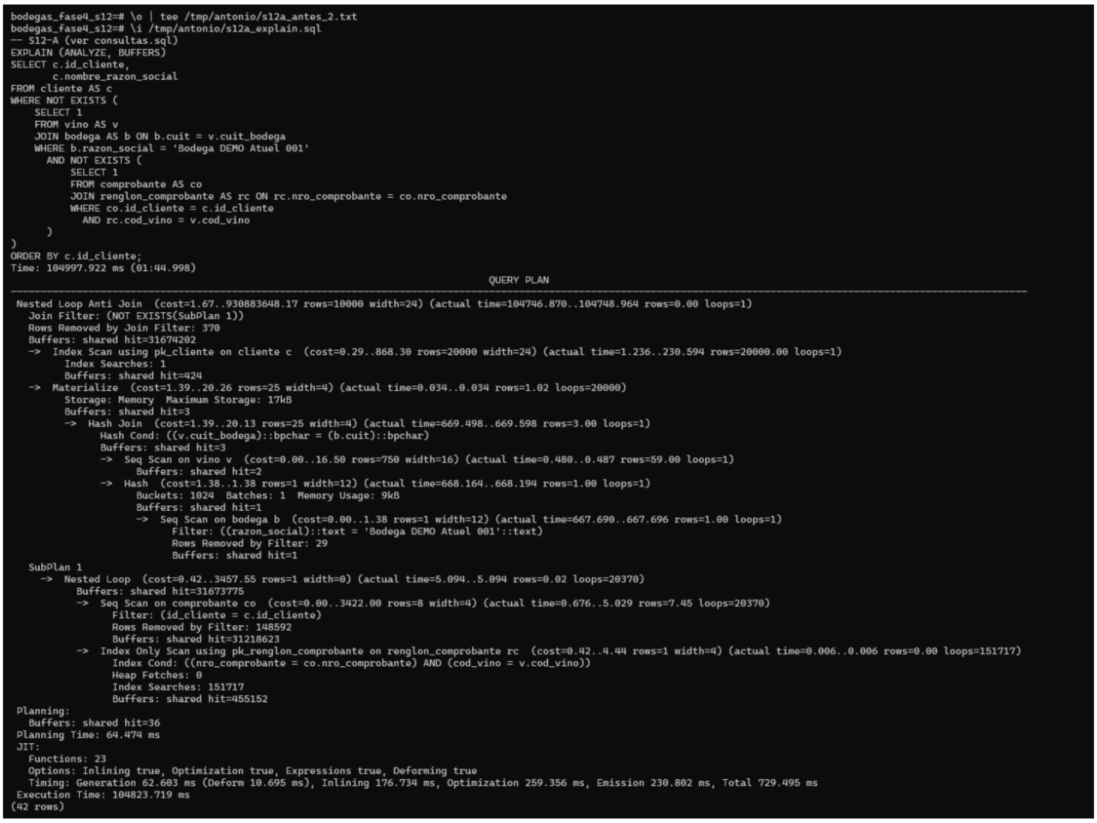
```
PS C:\Users\joseg\OneDrive\Documentos\3\Base de Datos\TRABAJO FINAL\trabajo_integrador_bd> Get-Content .\fase4\evidencias\jose\s4_despues_1.txt
                                                                                 QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=108.61..108.62 rows=5 width=12) (actual time=0.624..0.625 rows=5.00 loops=1)
   Buffers: shared hit=33 read=2
   ->  Sort  (cost=108.61..108.66 rows=20 width=12) (actual time=0.622..0.623 rows=5.00 loops=1)
         Sort Key: (sum(rt.cantidad)) DESC, t.id_sala
         Sort Method: top-N heapsort  Memory: 25kB
         Buffers: shared hit=33 read=2
         ->  GroupAggregate  (cost=107.93..108.28 rows=20 width=12) (actual time=0.586..0.591 rows=18.00 loops=1)
               Group Key: t.id_sala
               Buffers: shared hit=27 read=2
               ->  Sort  (cost=107.93..107.98 rows=20 width=8) (actual time=0.582..0.584 rows=20.00 loops=1)
                     Sort Key: t.id_sala
                     Sort Method: quicksort  Memory: 25kB
                     Buffers: shared hit=27 read=2
                     ->  Hash Join  (cost=41.24..107.50 rows=20 width=8) (actual time=0.170..0.571 rows=20.00 loops=1)
                           Hash Cond: (t.nro_solicitud = rt.nro_solicitud)
                           Buffers: shared hit=27 read=2
                           ->  Seq Scan on transferencia t  (cost=0.00..61.00 rows=2000 width=8) (actual time=0.014..0.347 rows=2000.00 loops=1)
                                 Filter: ((fecha_confeccion >= (CURRENT_DATE - '15 days'::interval)) AND (fecha_confeccion < (CURRENT_DATE + '1 day'::interval)))
                                 Buffers: shared hit=11
                           ->  Hash  (cost=40.99..40.99 rows=20 width=8) (actual time=0.120..0.120 rows=20.00 loops=1)
                                 Buckets: 1024  Batches: 1  Memory Usage: 9kB
                                 Buffers: shared hit=16 read=2
                                 ->  Bitmap Heap Scan on renglon_transferencia rt  (cost=4.44..40.99 rows=20 width=8) (actual time=0.054..0.110 rows=20.00 loops=1)
                                       Recheck Cond: (cod_vino = 681)
                                       Heap Blocks: exact=16
                                       Buffers: shared hit=16 read=2
                                       ->  Bitmap Index Scan on idx_rt_cod_vino_nro  (cost=0.00..4.43 rows=20 width=0) (actual time=0.043..0.043 rows=20.00 loops=1)
                                             Index Cond: (cod_vino = 681)
                                             Index Searches: 1
                                             Buffers: shared read=2
 Planning:
   Buffers: shared hit=161 read=1
 Planning Time: 0.894 ms
 Execution Time: 0.719 ms
(34 rows)
```

S4 después: acceso por idx_rt_cod_vino_nro, con 20 coincidencias.

4.3. S12-A – Clientes que compraron todos los vinos de una bodega

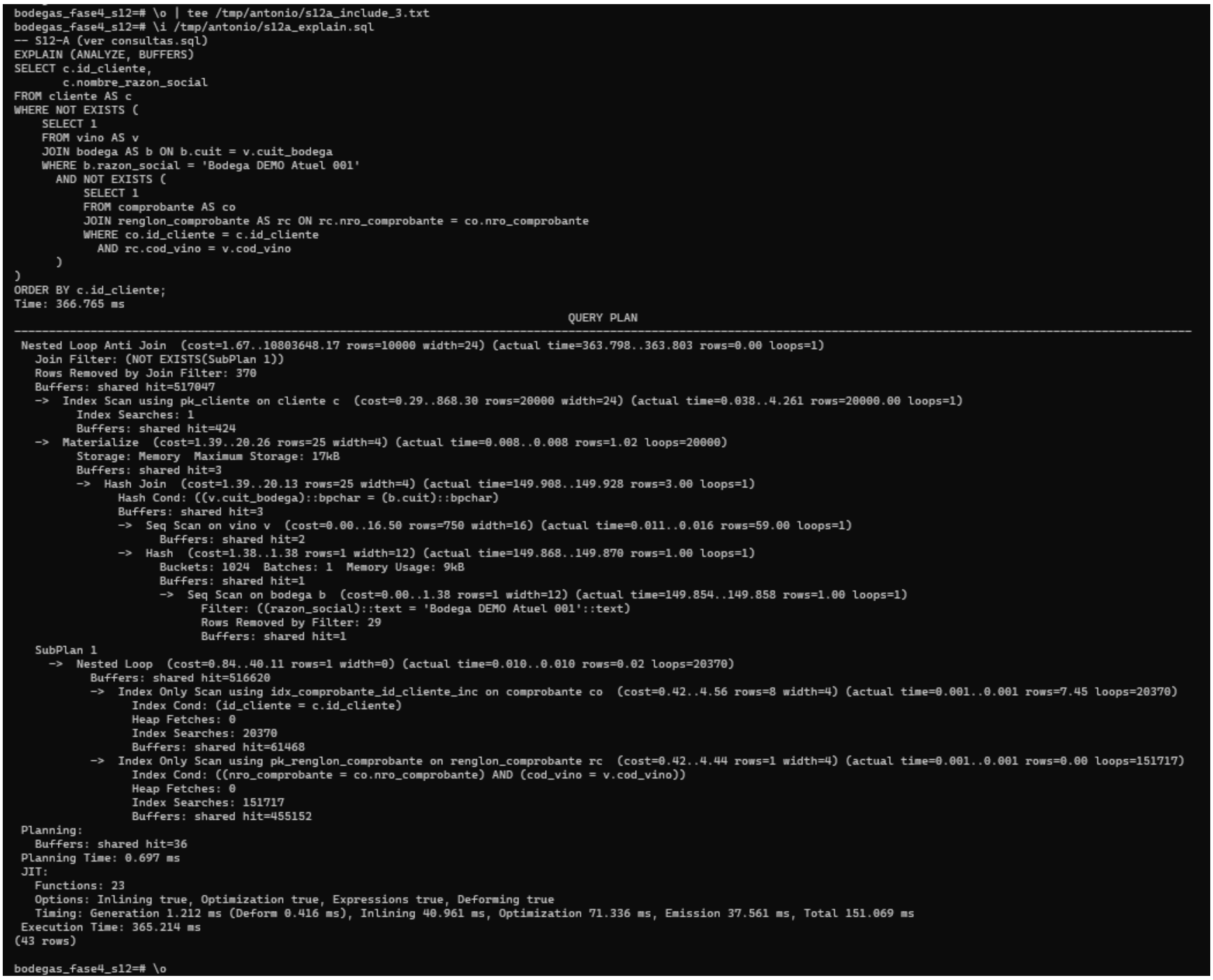
En S12-A buscamos los clientes que hubieran comprado todos los vinos de «Bodega DEMO Atuel 001», que tiene 34 vinos. La consulta utiliza una subconsulta que se repite para distintos clientes. Sin un índice por id_cliente, PostgreSQL recorría comprobante 20.370 veces. Con idx_comprobante_id_cliente_inc pudo buscar por cliente y obtener nro_comprobante directamente del índice, sin volver a leer esas filas de la tabla (Heap Fetches: 0).

El tiempo pasó de 104.823,719 ms a 365,214 ms, una mejora del 99,65 %. La consulta no encontró clientes que cumplieran la condición, pero la diferencia de rendimiento se observa igualmente en el trabajo que tuvo que realizar PostgreSQL.



```
bodegas_fase4_s12=# \o | tee /tmp/antonio/s12a_antes_2.txt
bodegas_fase4_s12=# \i /tmp/antonio/s12a_explain.sql
-- S12-A (ver consultas.sql)
EXPLAIN (ANALYZE, BUFFERS)
SELECT c.id_cliente,
       c.nombre_razon_social
FROM cliente AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM vino AS v
    JOIN bodega AS b ON b.cuit = v.cuit_bodega
    WHERE b.razon_social = 'Bodega DEMO Atuel 001'
      AND NOT EXISTS (
          SELECT 1
          FROM comprobante AS co
          JOIN renglon_comprobante AS rc ON rc.nro_comprobante = co.nro_comprobante
          WHERE co.id_cliente = c.id_cliente
            AND rc.cod_vino = v.cod_vino
      )
)
ORDER BY c.id_cliente;
Time: 104997.922 ms (01:44.998)
                                                                                 QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------------------------------
 Nested Loop Anti Join  (cost=1.67..930883648.17 rows=10000 width=24) (actual time=104746.870..104748.964 rows=0.00 loops=1)
   Join Filter: (NOT EXISTS(SubPlan 1))
   Rows Removed by Join Filter: 370
   Buffers: shared hit=31674202
   ->  Index Scan using pk_cliente on cliente c  (cost=0.29..868.30 rows=20000 width=24) (actual time=1.236..230.594 rows=20000.00 loops=1)
         Index Searches: 1
         Buffers: shared hit=424
   ->  Materialize  (cost=1.39..20.26 rows=25 width=4) (actual time=0.034..0.034 rows=1.02 loops=20000)
         Storage: Memory  Maximum Storage: 17kB
         Buffers: shared hit=3
         ->  Hash Join  (cost=1.39..20.13 rows=25 width=4) (actual time=669.498..669.598 rows=3.00 loops=1)
               Hash Cond: ((v.cuit_bodega)::bpchar = (b.cuit)::bpchar)
               Buffers: shared hit=3
               ->  Seq Scan on vino v  (cost=0.00..16.50 rows=750 width=16) (actual time=0.480..0.487 rows=59.00 loops=1)
                     Buffers: shared hit=2
               ->  Hash  (cost=1.38..1.38 rows=1 width=12) (actual time=668.164..668.194 rows=1.00 loops=1)
                     Buckets: 1024  Batches: 1  Memory Usage: 9kB
                     Buffers: shared hit=1
                     ->  Seq Scan on bodega b  (cost=0.00..1.38 rows=1 width=12) (actual time=667.690..667.696 rows=1.00 loops=1)
                           Filter: ((razon_social)::text = 'Bodega DEMO Atuel 001'::text)
                           Rows Removed by Filter: 29
                           Buffers: shared hit=1
   SubPlan 1
     ->  Nested Loop  (cost=0.42..3457.55 rows=1 width=0) (actual time=5.094..5.094 rows=0.02 loops=20370)
           Buffers: shared hit=31673775
           ->  Seq Scan on comprobante co  (cost=0.00..3422.00 rows=8 width=4) (actual time=0.676..5.029 rows=7.45 loops=20370)
                 Filter: (id_cliente = c.id_cliente)
                 Rows Removed by Filter: 148592
                 Buffers: shared hit=31218623
           ->  Index Only Scan using pk_renglon_comprobante on renglon_comprobante rc  (cost=0.42..4.44 rows=1 width=4) (actual time=0.006..0.006 rows=0.00 loops=151717)
                 Index Cond: ((nro_comprobante = co.nro_comprobante) AND (cod_vino = v.cod_vino))
                 Heap Fetches: 0
                 Index Searches: 151717
                 Buffers: shared hit=455152
 Planning:
   Buffers: shared hit=36
 Planning Time: 64.474 ms
 JIT:
   Functions: 23
   Options: Inlining true, Optimization true, Expressions true, Deforming true
   Timing: Generation 62.603 ms (Deform 10.695 ms), Inlining 176.734 ms, Optimization 259.356 ms, Emission 230.802 ms, Total 729.495 ms
 Execution Time: 104823.719 ms
(42 rows)
```

S12-A antes: acceso secuencial reiterado a comprobante en el subplan.



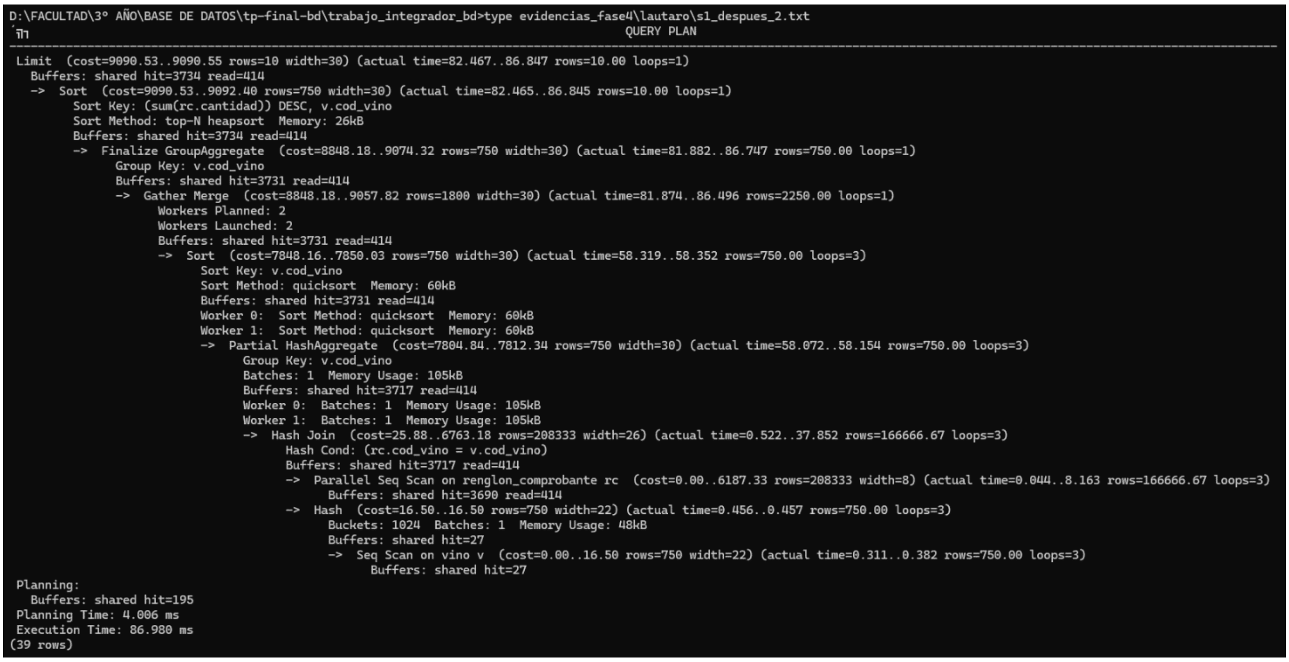
```
bodegas_fase4_s12=# \o | tee /tmp/antonio/s12a_include_3.txt
bodegas_fase4_s12=# \i /tmp/antonio/s12a_explain.sql
-- S12-A (ver consultas.sql)
EXPLAIN (ANALYZE, BUFFERS)
SELECT c.id_cliente,
       c.nombre_razon_social
FROM cliente AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM vino AS v
    JOIN bodega AS b ON b.cuit = v.cuit_bodega
    WHERE b.razon_social = 'Bodega DEMO Atuel 001'
      AND NOT EXISTS (
          SELECT 1
          FROM comprobante AS co
          JOIN renglon_comprobante AS rc ON rc.nro_comprobante = co.nro_comprobante
          WHERE co.id_cliente = c.id_cliente
            AND rc.cod_vino = v.cod_vino
      )
)
ORDER BY c.id_cliente;
Time: 366.765 ms
                                                                              QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------------------------------------------------
 Nested Loop Anti Join  (cost=1.67..10803648.17 rows=10000 width=24) (actual time=363.798..363.803 rows=0.00 loops=1)
   Join Filter: (NOT EXISTS(SubPlan 1))
   Rows Removed by Join Filter: 370
   Buffers: shared hit=517047
   ->  Index Scan using pk_cliente on cliente c  (cost=0.29..868.30 rows=20000 width=24) (actual time=0.038..4.261 rows=20000.00 loops=1)
         Index Searches: 1
         Buffers: shared hit=424
   ->  Materialize  (cost=1.39..20.26 rows=25 width=4) (actual time=0.008..0.008 rows=1.02 loops=20000)
         Storage: Memory  Maximum Storage: 17kB
         Buffers: shared hit=3
         ->  Hash Join  (cost=1.39..20.13 rows=25 width=4) (actual time=149.908..149.928 rows=3.00 loops=1)
               Hash Cond: ((v.cuit_bodega)::bpchar = (b.cuit)::bpchar)
               Buffers: shared hit=3
               ->  Seq Scan on vino v  (cost=0.00..16.50 rows=750 width=16) (actual time=0.011..0.016 rows=59.00 loops=1)
                     Buffers: shared hit=2
               ->  Hash  (cost=1.38..1.38 rows=1 width=12) (actual time=149.868..149.870 rows=1.00 loops=1)
                     Buckets: 1024  Batches: 1  Memory Usage: 9kB
                     Buffers: shared hit=1
                     ->  Seq Scan on bodega b  (cost=0.00..1.38 rows=1 width=12) (actual time=149.854..149.858 rows=1.00 loops=1)
                           Filter: ((razon_social)::text = 'Bodega DEMO Atuel 001'::text)
                           Rows Removed by Filter: 29
                           Buffers: shared hit=1
   SubPlan 1
     ->  Nested Loop  (cost=0.84..40.11 rows=1 width=0) (actual time=0.010..0.010 rows=0.02 loops=20370)
           Buffers: shared hit=516620
           ->  Index Only Scan using idx_comprobante_id_cliente_inc on comprobante co  (cost=0.42..4.56 rows=8 width=4) (actual time=0.001..0.001 rows=7.45 loops=20370)
                 Index Cond: (id_cliente = c.id_cliente)
                 Heap Fetches: 0
                 Index Searches: 20370
                 Buffers: shared hit=61468
           ->  Index Only Scan using pk_renglon_comprobante on renglon_comprobante rc  (cost=0.42..4.44 rows=1 width=4) (actual time=0.001..0.001 rows=0.00 loops=151717)
                 Index Cond: ((nro_comprobante = co.nro_comprobante) AND (cod_vino = v.cod_vino))
                 Heap Fetches: 0
                 Index Searches: 151717
                 Buffers: shared hit=455152
 Planning:
   Buffers: shared hit=36
 Planning Time: 0.697 ms
 JIT:
   Functions: 23
   Options: Inlining true, Optimization true, Expressions true, Deforming true
   Timing: Generation 1.212 ms (Deform 0.416 ms), Inlining 40.961 ms, Optimization 71.336 ms, Emission 37.561 ms, Total 151.069 ms
 Execution Time: 365.214 ms
(43 rows)

bodegas_fase4_s12=# \o
```

S12-A después: Index Only Scan con Heap Fetches: 0.

5. Índices descartados y casos sin utilización

5.1. S1 – Ranking global de vinos

En S1 probamos idx_rc_cod_vino para obtener el ranking general de vinos vendidos. El tiempo prácticamente no mejoró: pasó de 85,508 ms a 86,980 ms. PostgreSQL siguió recorriendo todos los renglones de venta, porque para calcular el ranking necesita considerar todas las ventas. Sin embargo, el índice sí fue útil en una búsqueda más específica del vino 442 (42,188 ms frente a 0,892 ms). Esa prueba adicional no se cuenta entre las tres consultas oficiales mejoradas.

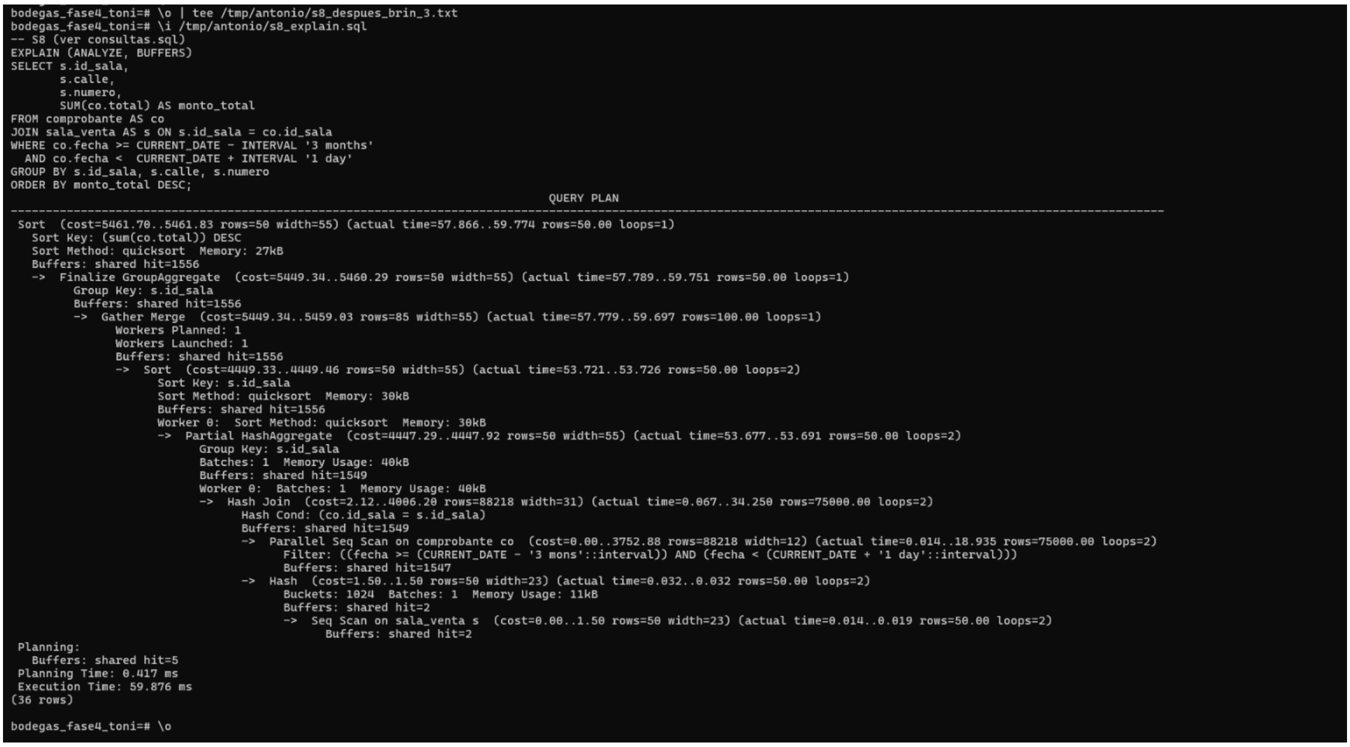


```
D:\FACULTAD\3° AÑO\BASE DE DATOS\tp-final-bd\trabajo_integrador_bd>type evidencias_fase4\lautaro\s1_despues_2.txt
´╗┐                                                                                        QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=9090.53..9090.55 rows=10 width=30) (actual time=82.467..86.847 rows=10.00 loops=1)
   Buffers: shared hit=3734 read=414
   ->  Sort  (cost=9090.53..9092.40 rows=750 width=30) (actual time=82.465..86.845 rows=10.00 loops=1)
         Sort Key: (sum(rc.cantidad)) DESC, v.cod_vino
         Sort Method: top-N heapsort  Memory: 26kB
         Buffers: shared hit=3734 read=414
         ->  Finalize GroupAggregate  (cost=8848.18..9074.32 rows=750 width=30) (actual time=81.882..86.747 rows=750.00 loops=1)
               Group Key: v.cod_vino
               Buffers: shared hit=3731 read=414
               ->  Gather Merge  (cost=8848.18..9057.82 rows=1800 width=30) (actual time=81.874..86.496 rows=2250.00 loops=1)
                     Workers Planned: 2
                     Workers Launched: 2
                     Buffers: shared hit=3731 read=414
                     ->  Sort  (cost=7848.16..7850.03 rows=750 width=30) (actual time=58.319..58.352 rows=750.00 loops=3)
                           Sort Key: v.cod_vino
                           Sort Method: quicksort  Memory: 60kB
                           Buffers: shared hit=3731 read=414
                           Worker 0:  Sort Method: quicksort  Memory: 60kB
                           Worker 1:  Sort Method: quicksort  Memory: 60kB
                           ->  Partial HashAggregate  (cost=7804.84..7812.34 rows=750 width=30) (actual time=58.072..58.154 rows=750.00 loops=3)
                                 Group Key: v.cod_vino
                                 Batches: 1  Memory Usage: 105kB
                                 Buffers: shared hit=3717 read=414
                                 Worker 0:  Batches: 1  Memory Usage: 105kB
                                 Worker 1:  Batches: 1  Memory Usage: 105kB
                                 ->  Hash Join  (cost=25.88..6763.18 rows=208333 width=26) (actual time=0.522..37.852 rows=166666.67 loops=3)
                                       Hash Cond: (rc.cod_vino = v.cod_vino)
                                       Buffers: shared hit=3717 read=414
                                       ->  Parallel Seq Scan on renglon_comprobante rc  (cost=0.00..6187.33 rows=208333 width=8) (actual time=0.044..8.163 rows=166666.67 loops=3)
                                             Buffers: shared hit=3690 read=414
                                       ->  Hash  (cost=16.50..16.50 rows=750 width=22) (actual time=0.456..0.457 rows=750.00 loops=3)
                                             Buckets: 1024  Batches: 1  Memory Usage: 48kB
                                             Buffers: shared hit=27
                                             ->  Seq Scan on vino v  (cost=0.00..16.50 rows=750 width=22) (actual time=0.311..0.382 rows=750.00 loops=3)
                                                   Buffers: shared hit=27
 Planning:
   Buffers: shared hit=195
 Planning Time: 4.006 ms
 Execution Time: 86.980 ms
(39 rows)
```

S1 con índice existente: el planificador continúa eligiendo Parallel Seq Scan.

5.2. S8 – INCLUDE y BRIN evaluados y descartados

En S8 probamos un índice con INCLUDE y otro BRIN sobre la fecha. Ninguno fue utilizado: PostgreSQL mantuvo el recorrido secuencial porque los 150.000 comprobantes entraban en el período consultado. Aunque algunas ejecuciones fueron más rápidas, también mejoraron las mediciones sin índice. Por eso no atribuimos esa diferencia a los índices y decidimos descartarlos. El índice con INCLUDE, además, era parecido al índice compuesto ya aprobado para S3.

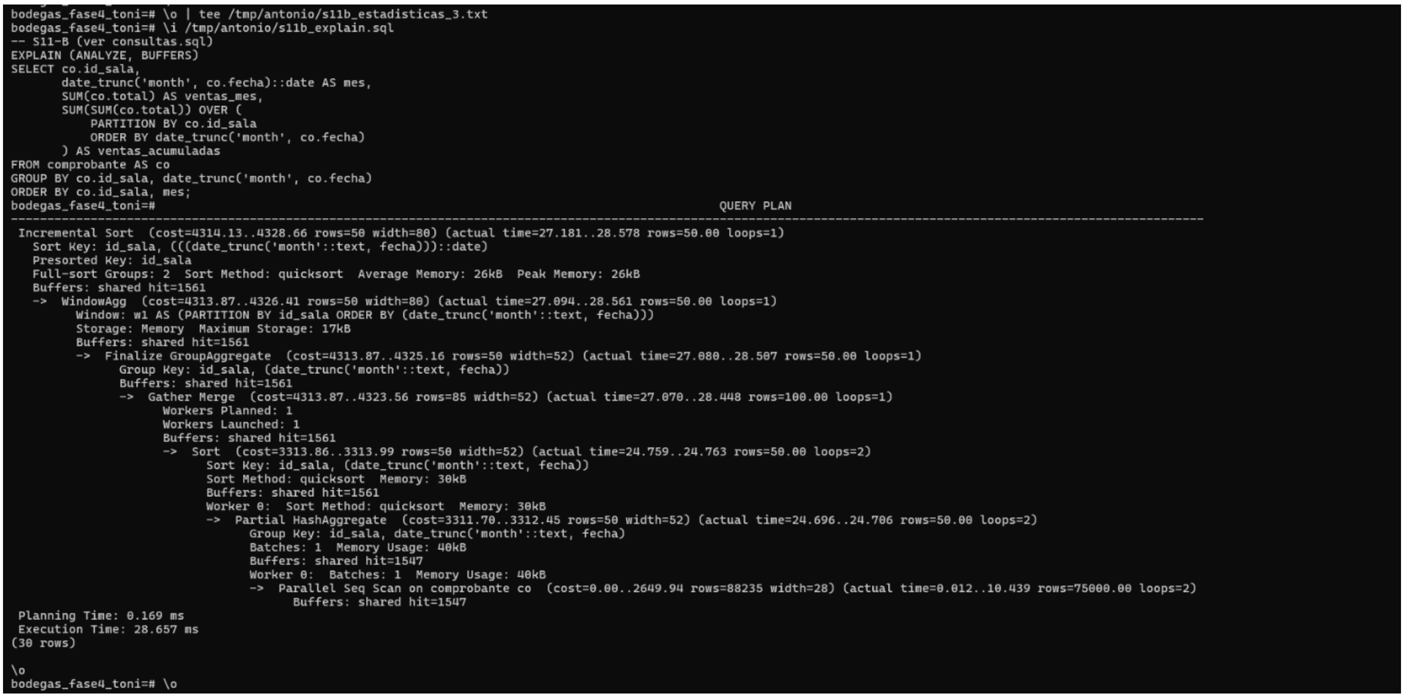


```
bodegas_fase4_toni=# \o | tee /tmp/antonio/s8_despues_brin_3.txt
bodegas_fase4_toni=# \i /tmp/antonio/s8_explain.sql
-- S8 (ver consultas.sql)
EXPLAIN (ANALYZE, BUFFERS)
SELECT s.id_sala,
       s.calle,
       s.numero,
       SUM(co.total) AS monto_total
FROM comprobante AS co
JOIN sala_venta AS s ON s.id_sala = co.id_sala
WHERE co.fecha >= CURRENT_DATE - INTERVAL '3 months'
  AND co.fecha <  CURRENT_DATE + INTERVAL '1 day'
GROUP BY s.id_sala, s.calle, s.numero
ORDER BY monto_total DESC;
                                                                               QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=5461.70..5461.83 rows=50 width=55) (actual time=57.866..59.774 rows=50.00 loops=1)
   Sort Key: (sum(co.total)) DESC
   Sort Method: quicksort  Memory: 27kB
   Buffers: shared hit=1556
   ->  Finalize GroupAggregate  (cost=5449.34..5460.29 rows=50 width=55) (actual time=57.789..59.751 rows=50.00 loops=1)
         Group Key: s.id_sala
         Buffers: shared hit=1556
         ->  Gather Merge  (cost=5449.34..5459.03 rows=85 width=55) (actual time=57.779..59.697 rows=100.00 loops=1)
               Workers Planned: 1
               Workers Launched: 1
               Buffers: shared hit=1556
               ->  Sort  (cost=4449.33..4449.46 rows=50 width=55) (actual time=53.721..53.726 rows=50.00 loops=2)
                     Sort Key: s.id_sala
                     Sort Method: quicksort  Memory: 30kB
                     Buffers: shared hit=1556
                     Worker 0:  Sort Method: quicksort  Memory: 30kB
                     ->  Partial HashAggregate  (cost=4447.29..4447.92 rows=50 width=55) (actual time=53.677..53.691 rows=50.00 loops=2)
                           Group Key: s.id_sala
                           Batches: 1  Memory Usage: 40kB
                           Buffers: shared hit=1549
                           Worker 0:  Batches: 1  Memory Usage: 40kB
                           ->  Hash Join  (cost=2.12..4006.20 rows=88218 width=31) (actual time=0.067..34.250 rows=75000.00 loops=2)
                                 Hash Cond: (co.id_sala = s.id_sala)
                                 Buffers: shared hit=1549
                                 ->  Parallel Seq Scan on comprobante co  (cost=0.00..3752.88 rows=88218 width=12) (actual time=0.014..18.935 rows=75000.00 loops=2)
                                       Filter: ((fecha >= (CURRENT_DATE - '3 mons'::interval)) AND (fecha < (CURRENT_DATE + '1 day'::interval)))
                                       Buffers: shared hit=1547
                                 ->  Hash  (cost=1.50..1.50 rows=50 width=23) (actual time=0.032..0.032 rows=50.00 loops=2)
                                       Buckets: 1024  Batches: 1  Memory Usage: 11kB
                                       Buffers: shared hit=2
                                       ->  Seq Scan on sala_venta s  (cost=0.00..1.50 rows=50 width=23) (actual time=0.014..0.019 rows=50.00 loops=2)
                                             Buffers: shared hit=2
 Planning:
   Buffers: shared hit=5
 Planning Time: 0.417 ms
 Execution Time: 59.876 ms
(36 rows)

bodegas_fase4_toni=# \o
```

S8: PostgreSQL no utiliza el BRIN existente; conserva el escaneo secuencial.

5.3. S11-B – Diferencia atribuible a estadísticas, no al indice

En S11-B, el tiempo bajó de 47,448 a 28,259 ms después de crear un índice de expresión, pero el plan no usaba ese índice. Al quitarlo y actualizar las estadísticas sobre la misma expresión, se obtuvo un tiempo similar (28,657 ms). Concluimos que la mejora no provenía de leer el índice, así que también lo descartamos.

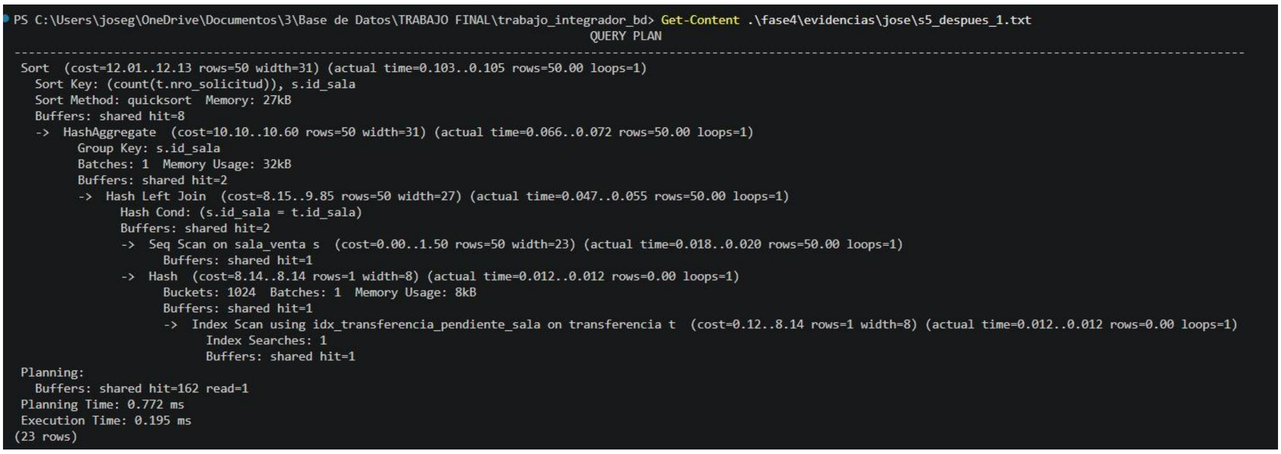


```
bodegas_fase4_toni=# \o | tee /tmp/antonio/s11b_estadisticas_3.txt
bodegas_fase4_toni=# \i /tmp/antonio/s11b_explain.sql
-- S11-B (ver consultas.sql)
EXPLAIN (ANALYZE, BUFFERS)
SELECT co.id_sala,
       date_trunc('month', co.fecha)::date AS mes,
       SUM(co.total) AS ventas_mes,
       SUM(SUM(co.total)) OVER (
           PARTITION BY co.id_sala
           ORDER BY date_trunc('month', co.fecha)
       ) AS ventas_acumuladas
FROM comprobante AS co
GROUP BY co.id_sala, date_trunc('month', co.fecha)
ORDER BY co.id_sala, mes;
bodegas_fase4_toni=#                                                                                                  QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------------------------------------
 Incremental Sort  (cost=4314.13..4328.66 rows=50 width=80) (actual time=27.181..28.578 rows=50.00 loops=1)
   Sort Key: id_sala, (((date_trunc('month'::text, fecha)))::date)
   Presorted Key: id_sala
   Full-sort Groups: 2  Sort Method: quicksort  Average Memory: 26kB  Peak Memory: 26kB
   Buffers: shared hit=1561
   ->  WindowAgg  (cost=4313.87..4326.41 rows=50 width=80) (actual time=27.094..28.561 rows=50.00 loops=1)
         Window: w1 AS (PARTITION BY id_sala ORDER BY (date_trunc('month'::text, fecha)))
         Storage: Memory  Maximum Storage: 17kB
         Buffers: shared hit=1561
         ->  Finalize GroupAggregate  (cost=4313.87..4325.16 rows=50 width=52) (actual time=27.080..28.507 rows=50.00 loops=1)
               Group Key: id_sala, (date_trunc('month'::text, fecha))
               Buffers: shared hit=1561
               ->  Gather Merge  (cost=4313.87..4323.56 rows=85 width=52) (actual time=27.070..28.448 rows=100.00 loops=1)
                     Workers Planned: 1
                     Workers Launched: 1
                     Buffers: shared hit=1561
                     ->  Sort  (cost=3313.86..3313.99 rows=50 width=52) (actual time=24.759..24.763 rows=50.00 loops=2)
                           Sort Key: id_sala, (date_trunc('month'::text, fecha))
                           Sort Method: quicksort  Memory: 30kB
                           Buffers: shared hit=1561
                           Worker 0:  Sort Method: quicksort  Memory: 30kB
                           ->  Partial HashAggregate  (cost=3311.70..3312.45 rows=50 width=52) (actual time=24.696..24.706 rows=50.00 loops=2)
                                 Group Key: id_sala, date_trunc('month'::text, fecha)
                                 Batches: 1  Memory Usage: 40kB
                                 Buffers: shared hit=1547
                                 Worker 0:  Batches: 1  Memory Usage: 40kB
                                 ->  Parallel Seq Scan on comprobante co  (cost=0.00..2649.94 rows=88235 width=28) (actual time=0.012..10.439 rows=75000.00 loops=2)
                                       Buffers: shared hit=1547
 Planning Time: 0.169 ms
 Execution Time: 28.657 ms
(30 rows)

\o
bodegas_fase4_toni=# \o
```

S11-B: las estadísticas de la expresión reproducen el plan sin recorrer el índice.

5.4. S5 – Indice parcial con conjunto vacío

El índice parcial idx_transferencia_pendiente_sala se creó para las transferencias cuyo estado es distinto de 'procesado'. PostgreSQL lo utilizó y el tiempo pasó de 0,311 a 0,198 ms. Sin embargo, las 2.000 transferencias del dump ya estaban procesadas, por lo que no había pendientes. Por ese motivo lo conservamos como índice parcial válido, pero no contamos S5 entre las tres mejoras principales.



```
PS C:\Users\joseg\OneDrive\Documentos\3\Base de Datos\TRABAJO FINAL\trabajo_integrador_bd> Get-Content .\fase4\evidencias\jose\s5_despues_1.txt
                                                                                      QUERY PLAN
----------------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=12.01..12.13 rows=50 width=31) (actual time=0.103..0.105 rows=50.00 loops=1)
   Sort Key: (count(t.nro_solicitud)), s.id_sala
   Sort Method: quicksort  Memory: 27kB
   Buffers: shared hit=8
   ->  HashAggregate  (cost=10.10..10.60 rows=50 width=31) (actual time=0.066..0.072 rows=50.00 loops=1)
         Group Key: s.id_sala
         Batches: 1  Memory Usage: 32kB
         Buffers: shared hit=2
         ->  Hash Left Join  (cost=8.15..9.85 rows=50 width=27) (actual time=0.047..0.055 rows=50.00 loops=1)
               Hash Cond: (s.id_sala = t.id_sala)
               Buffers: shared hit=2
               ->  Seq Scan on sala_venta s  (cost=0.00..1.50 rows=50 width=23) (actual time=0.018..0.020 rows=50.00 loops=1)
                     Buffers: shared hit=1
               ->  Hash  (cost=8.14..8.14 rows=1 width=8) (actual time=0.012..0.012 rows=0.00 loops=1)
                     Buckets: 1024  Batches: 1  Memory Usage: 8kB
                     Buffers: shared hit=1
                     ->  Index Scan using idx_transferencia_pendiente_sala on transferencia t  (cost=0.12..8.14 rows=1 width=8) (actual time=0.012..0.012 rows=0.00 loops=1)
                           Index Searches: 1
                           Buffers: shared hit=1
 Planning:
   Buffers: shared hit=162 read=1
 Planning Time: 0.772 ms
 Execution Time: 0.195 ms
(23 rows)
```

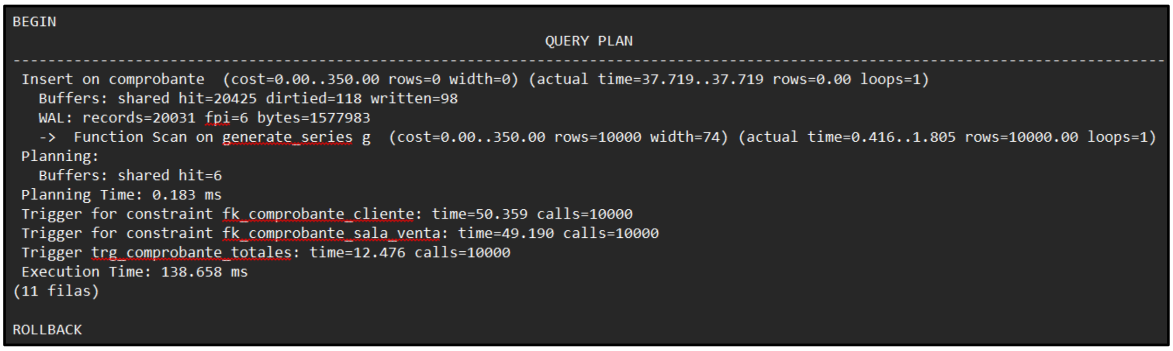
S5: utilización del índice parcial con cero filas pendientes en el dataset.

6. Costo de escritura: Prueba con 10.000 comprobantes

Fabri comparó la inserción de 10.000 cabeceras de comprobantes en dos copias de la misma base: una con los índices originales y otra con dos índices adicionales sobre comprobante. Mantuvo activas las restricciones y los triggers. Hizo tres pruebas por escenario, utilizando BEGIN y ROLLBACK para no dejar registros insertados.

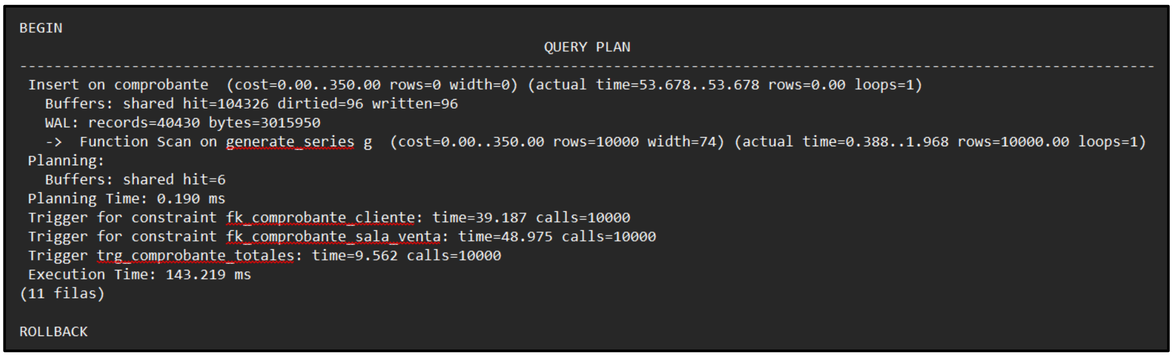
Repetición	Sin índices nuevos (ms)	Con dos índices nuevos (ms)
1	138,658	236,857
2	151,735	142,877
3	123,761	143,219
Mediana	138,658	143,219

La mediana pasó de 138,658 ms sin índices adicionales a 143,219 ms con ellos. Eso representa un sobrecosto observado del 3,29 %. La diferencia fue pequeña y los tiempos variaron entre repeticiones; por eso no se puede afirmar que agregar índices siempre aumente el tiempo en ese porcentaje. Esta prueba mide cabeceras de comprobantes, no ventas completas.



```
BEGIN
                                                                    QUERY PLAN
--------------------------------------------------------------------------------------------------------------------------------------
 Insert on comprobante  (cost=0.00..350.00 rows=0 width=0) (actual time=37.719..37.719 rows=0.00 loops=1)
   Buffers: shared hit=20425 dirtied=118 written=98
   WAL: records=20031 fpi=6 bytes=1577983
   ->  Function Scan on generate_series g  (cost=0.00..350.00 rows=10000 width=74) (actual time=0.416..1.805 rows=10000.00 loops=1)
 Planning:
   Buffers: shared hit=6
 Planning Time: 0.183 ms
 Trigger for constraint fk_comprobante_cliente: time=50.359 calls=10000
 Trigger for constraint fk_comprobante_sala_venta: time=49.190 calls=10000
 Trigger trg_comprobante_totales: time=12.476 calls=10000
 Execution Time: 138.658 ms
(11 filas)

ROLLBACK
```

Prueba de escrituras: ejecución sin los dos índices adicionales.



```
BEGIN
                                                                    QUERY PLAN
--------------------------------------------------------------------------------------------------------------------------------------
 Insert on comprobante  (cost=0.00..350.00 rows=0 width=0) (actual time=53.678..53.678 rows=0.00 loops=1)
   Buffers: shared hit=104326 dirtied=96 written=96
   WAL: records=40430 bytes=3015950
   ->  Function Scan on generate_series g  (cost=0.00..350.00 rows=10000 width=74) (actual time=0.388..1.968 rows=10000.00 loops=1)
 Planning:
   Buffers: shared hit=6
 Planning Time: 0.190 ms
 Trigger for constraint fk_comprobante_cliente: time=39.187 calls=10000
 Trigger for constraint fk_comprobante_sala_venta: time=48.975 calls=10000
 Trigger trg_comprobante_totales: time=9.562 calls=10000
 Execution Time: 143.219 ms
(11 filas)

ROLLBACK
```

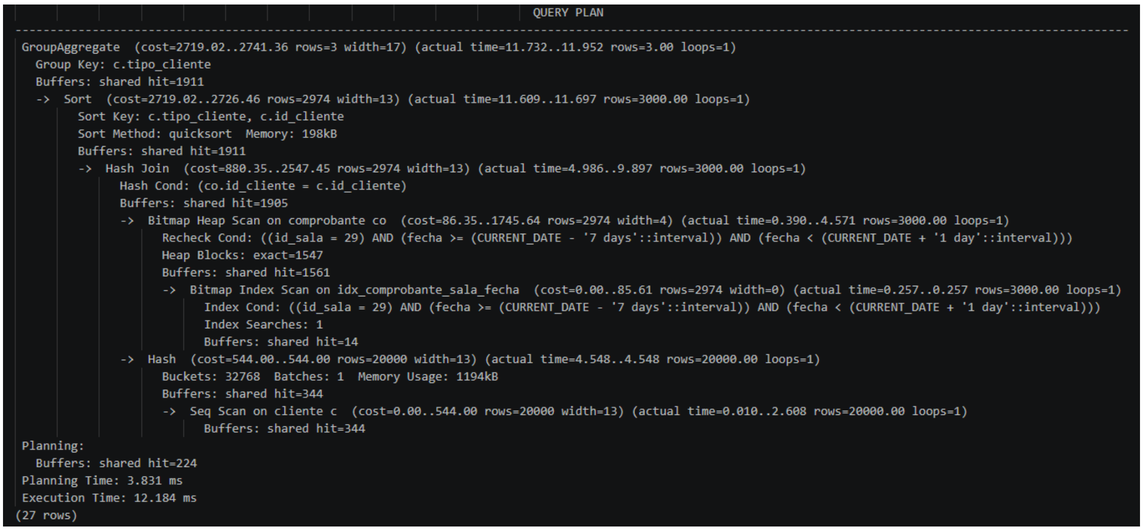
Prueba de escrituras: ejecución con los dos índices adicionales.

7. Validación conjunta de la migración 008_indices.sql

Para la integración, Lautaro restauró otra base de prueba y ejecutó 008_indices.sql con los cinco índices. Verificó sus definiciones en pg_indexes, confirmó los 38 índices y actualizó las estadísticas. Después repitió tres veces S3, S4 y S12-A con los mismos parámetros utilizados anteriormente.

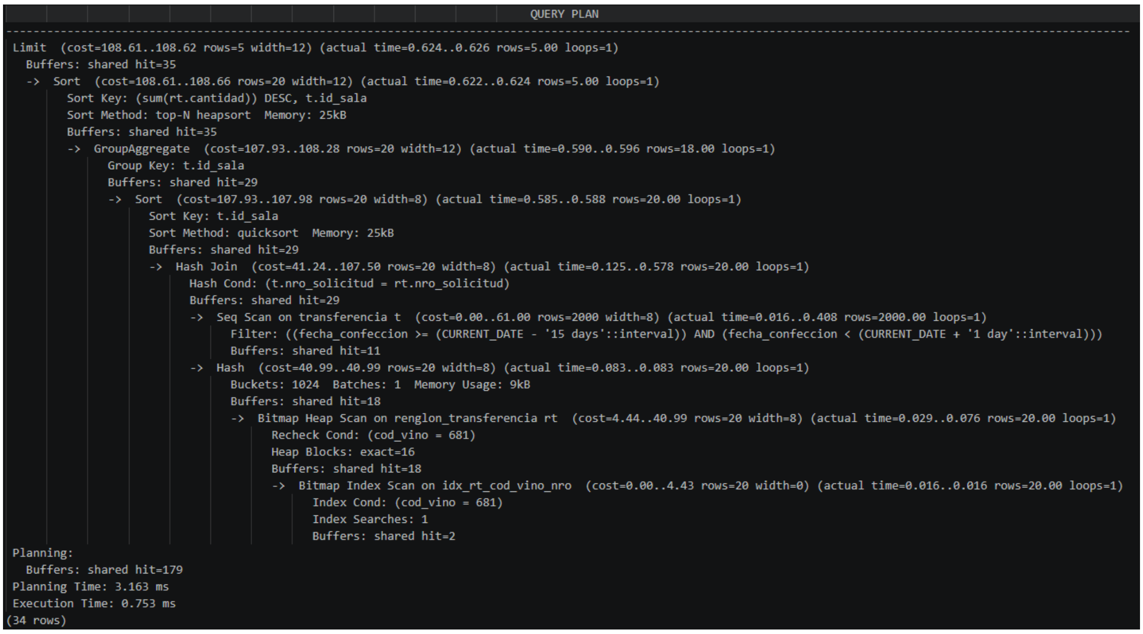
Consulta	Mediana de integración	Acceso con los cinco índices instalados
S3	12,344 ms	Bitmap Index Scan – idx_comprobante_sala_fecha
S4	0,757 ms	Bitmap Index Scan – idx_rt_cod_vino_nro
S12-A	251,774 ms	Index Only Scan – idx_comprobante_id_cliente_inc; Heap Fetches: 0

Los resultados confirman que PostgreSQL sigue utilizando los índices de S3, S4 y S12-A cuando están los cinco instalados. Estas mediciones solo comprueban el funcionamiento conjunto: las comparaciones antes y después son las de las pruebas individuales.



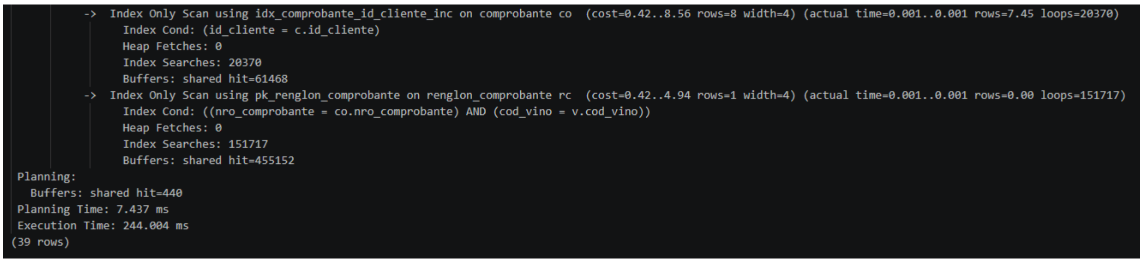
```
                                                  QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------------
 GroupAggregate  (cost=2719.02..2741.36 rows=3 width=17) (actual time=11.732..11.952 rows=3.00 loops=1)
   Group Key: c.tipo_cliente
   Buffers: shared hit=1911
   ->  Sort  (cost=2719.02..2726.46 rows=2974 width=13) (actual time=11.609..11.697 rows=3000.00 loops=1)
         Sort Key: c.tipo_cliente, c.id_cliente
         Sort Method: quicksort  Memory: 198kB
         Buffers: shared hit=1911
         ->  Hash Join  (cost=880.35..2547.45 rows=2974 width=13) (actual time=4.986..9.897 rows=3000.00 loops=1)
               Hash Cond: (co.id_cliente = c.id_cliente)
               Buffers: shared hit=1905
               ->  Bitmap Heap Scan on comprobante co  (cost=86.35..1745.64 rows=2974 width=4) (actual time=0.390..4.571 rows=3000.00 loops=1)
                     Recheck Cond: ((id_sala = 29) AND (fecha >= (CURRENT_DATE - '7 days'::interval)) AND (fecha < (CURRENT_DATE + '1 day'::interval)))
                     Heap Blocks: exact=1547
                     Buffers: shared hit=1561
                     ->  Bitmap Index Scan on idx_comprobante_sala_fecha  (cost=0.00..85.61 rows=2974 width=0) (actual time=0.257..0.257 rows=3000.00 loops=1)
                           Index Cond: ((id_sala = 29) AND (fecha >= (CURRENT_DATE - '7 days'::interval)) AND (fecha < (CURRENT_DATE + '1 day'::interval)))
                           Index Searches: 1
                           Buffers: shared hit=14
               ->  Hash  (cost=544.00..544.00 rows=20000 width=13) (actual time=4.548..4.548 rows=20000.00 loops=1)
                     Buckets: 32768  Batches: 1  Memory Usage: 1194kB
                     Buffers: shared hit=344
                     ->  Seq Scan on cliente c  (cost=0.00..544.00 rows=20000 width=13) (actual time=0.010..2.608 rows=20000.00 loops=1)
                           Buffers: shared hit=344
 Planning:
   Buffers: shared hit=224
 Planning Time: 3.831 ms
 Execution Time: 12.184 ms
(27 rows)
```

Integración: S3 usa el índice compuesto con los cinco índices ya instalados.



```
                                                  QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------------
Limit  (cost=108.61..108.62 rows=5 width=12) (actual time=0.624..0.626 rows=5.00 loops=1)
  Buffers: shared hit=35
  ->  Sort  (cost=108.61..108.66 rows=20 width=12) (actual time=0.622..0.624 rows=5.00 loops=1)
        Sort Key: (sum(rt.cantidad)) DESC, t.id_sala
        Sort Method: top-N heapsort  Memory: 25kB
        Buffers: shared hit=35
        ->  GroupAggregate  (cost=107.93..108.28 rows=20 width=12) (actual time=0.590..0.596 rows=18.00 loops=1)
              Group Key: t.id_sala
              Buffers: shared hit=29
              ->  Sort  (cost=107.93..107.98 rows=20 width=8) (actual time=0.585..0.588 rows=20.00 loops=1)
                    Sort Key: t.id_sala
                    Sort Method: quicksort  Memory: 25kB
                    Buffers: shared hit=29
                    ->  Hash Join  (cost=41.24..107.50 rows=20 width=8) (actual time=0.125..0.578 rows=20.00 loops=1)
                          Hash Cond: (t.nro_solicitud = rt.nro_solicitud)
                          Buffers: shared hit=29
                          ->  Seq Scan on transferencia t  (cost=0.00..61.00 rows=2000 width=8) (actual time=0.016..0.408 rows=2000.00 loops=1)
                                Filter: ((fecha_confeccion >= (CURRENT_DATE - '15 days'::interval)) AND (fecha_confeccion < (CURRENT_DATE + '1 day'::interval)))
                                Buffers: shared hit=11
                          ->  Hash  (cost=40.99..40.99 rows=20 width=8) (actual time=0.083..0.083 rows=20.00 loops=1)
                                Buckets: 1024  Batches: 1  Memory Usage: 9kB
                                Buffers: shared hit=18
                                ->  Bitmap Heap Scan on renglon_transferencia rt  (cost=4.44..40.99 rows=20 width=8) (actual time=0.029..0.076 rows=20.00 loops=1)
                                      Recheck Cond: (cod_vino = 681)
                                      Heap Blocks: exact=16
                                      Buffers: shared hit=18
                                      ->  Bitmap Index Scan on idx_rt_cod_vino_nro  (cost=0.00..4.43 rows=20 width=0) (actual time=0.016..0.016 rows=20.00 loops=1)
                                            Index Cond: (cod_vino = 681)
                                            Index Searches: 1
                                            Buffers: shared hit=2
Planning:
  Buffers: shared hit=179
Planning Time: 3.163 ms
Execution Time: 0.753 ms
(34 rows)
```

Integración: S4 usa idx_rt_cod_vino_nro.



```
                    ->  Index Only Scan using idx_comprobante_id_cliente_inc on comprobante co  (cost=0.42..8.56 rows=8 width=4) (actual time=0.001..0.001 rows=7.45 loops=20370)
                          Index Cond: (id_cliente = c.id_cliente)
                          Heap Fetches: 0
                          Index Searches: 20370
                          Buffers: shared hit=61468
                    ->  Index Only Scan using pk_renglon_comprobante on renglon_comprobante rc  (cost=0.42..4.94 rows=1 width=4) (actual time=0.001..0.001 rows=0.00 loops=151717)
                          Index Cond: ((nro_comprobante = co.nro_comprobante) AND (cod_vino = v.cod_vino))
                          Heap Fetches: 0
                          Index Searches: 151717
                          Buffers: shared hit=455152
 Planning:
   Buffers: shared hit=440
 Planning Time: 7.437 ms
 Execution Time: 244.004 ms
(39 rows)
```

Integración: S12-A realiza Index Only Scan con Heap Fetches: 0.

8. Limitaciones de los resultados

Las pruebas se hicieron en equipos diferentes, por lo que los tiempos pueden variar según la computadora y el estado de la base. Por eso usamos tres repeticiones y revisamos los planes, además del tiempo. En S8 y S11-B comprobamos que una consulta puede tardar menos sin que la causa sea el índice probado.

También encontramos dos limitaciones en los datos: todos los comprobantes comerciales tienen la misma fecha y no hay transferencias pendientes. Además, S12-A no devolvió clientes para la bodega elegida. Estos resultados sirven para evaluar las consultas probadas, pero no garantizan el mismo comportamiento con otros datos.

9. Conclusión técnica

En la Fase 4 creamos y probamos cinco índices de los tipos pedidos: compuesto, parcial, sobre clave foránea y especial con INCLUDE. Comprobamos mejoras en tres consultas oficiales: S3 (64,26 %), S4 (30,93 %) y S12-A (99,65 %). También explicamos por qué descartamos algunos índices y mostramos casos donde PostgreSQL decidió no utilizarlos.

La prueba final confirmó que los cinco índices estaban creados y que los correspondientes a S3, S4 y S12-A seguían utilizándose. En el ensayo de inserción observamos un 3,29 % más de tiempo al mantener dos índices adicionales. Esto nos mostró que un índice puede ayudar en las búsquedas, pero también tiene un costo al insertar datos; por eso es importante probarlo antes de incorporarlo.

10. Fuentes y ubicación de evidencias

Nos basamos en la consigna oficial «TPI Bases de Datos — Bodegas del Atuel», Etapa 4 (páginas 14 y 15), y en las consultas de la sección 8. El archivo de índices definitivo es db/migrations/008_indices.sql y los datos originales proceden del respaldo full de la Fase 3.

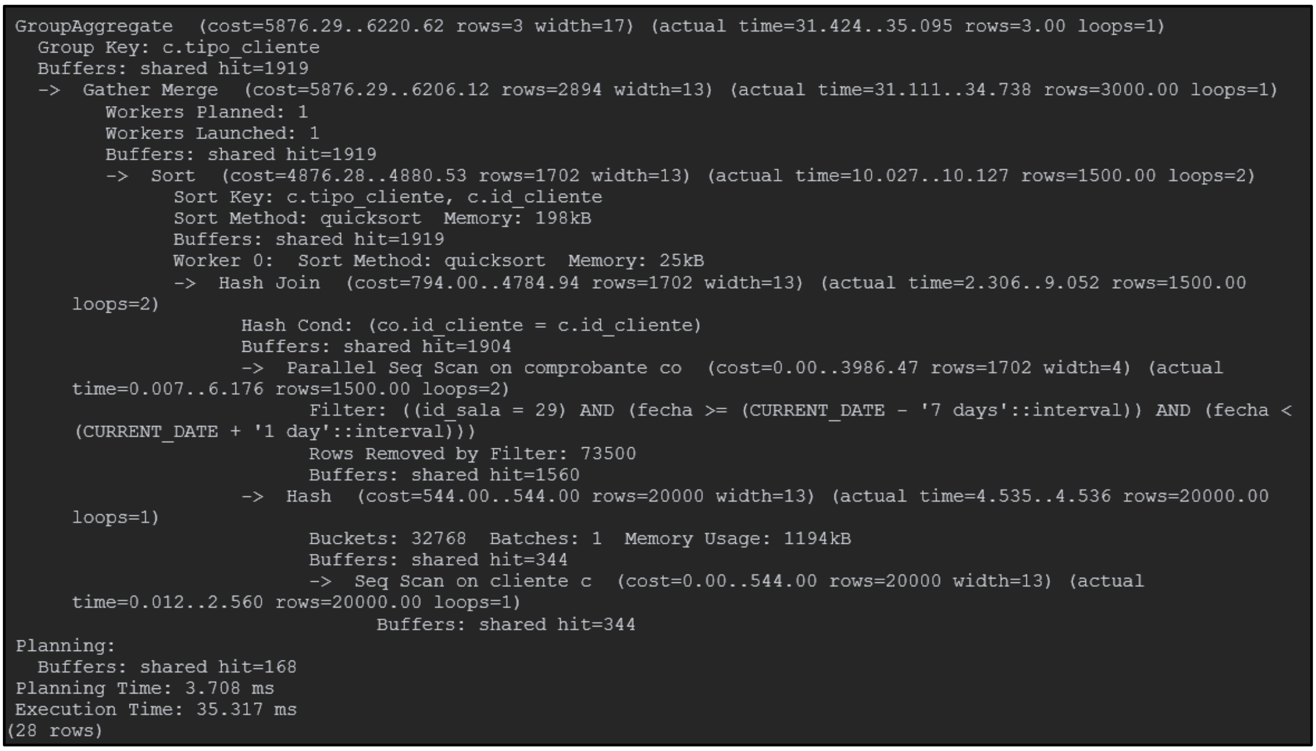
Los archivos originales quedaron organizados en fase4/evidencias/lautaro/, fase4/evidencias/montana/ (Antonio), fase4/evidencias/jose/, fase4/evidencias/fabri/ y fase4/evidencias/integracion/. Allí se conservan las consultas, los reportes, las capturas y las tres repeticiones de cada prueba. En el anexo incluimos planes completos seleccionados que respaldan las mejoras principales y las decisiones de descarte.

Anexo – Planes completos de ejecución

Se incluyen los planes completos de las tres consultas principales, más ejemplos de índices descartados o no utilizados. Son los registros originales de PostgreSQL; las demás repeticiones se conservan en las carpetas de evidencias del equipo. Las capturas ya aparecen en el desarrollo y no se vuelven a insertar aquí.

A1. S3 — antes del índice

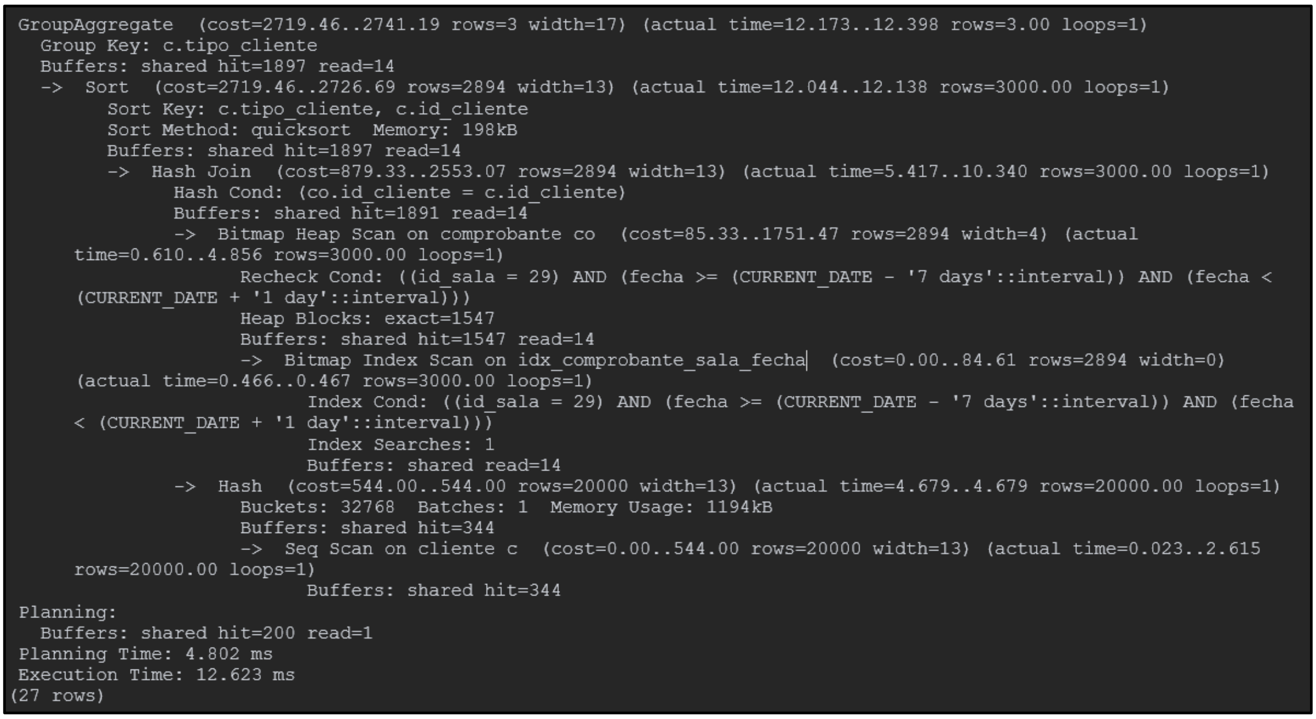
Lautaro · 35,317 ms · Archivo: s3_antes_2.txt



```
 GroupAggregate  (cost=5876.29..6220.62 rows=3 width=17) (actual time=31.424..35.095 rows=3.00 loops=1)
   Group Key: c.tipo_cliente
   Buffers: shared hit=1919
   ->  Gather Merge  (cost=5876.29..6206.12 rows=2894 width=13) (actual time=31.111..34.738 rows=3000.00 loops=1)
         Workers Planned: 1
         Workers Launched: 1
         Buffers: shared hit=1919
         ->  Sort  (cost=4876.28..4880.53 rows=1702 width=13) (actual time=10.027..10.127 rows=1500.00 loops=2)
               Sort Key: c.tipo_cliente, c.id_cliente
               Sort Method: quicksort  Memory: 198kB
               Buffers: shared hit=1919
               Worker 0:  Sort Method: quicksort  Memory: 25kB
               ->  Hash Join  (cost=794.00..4784.94 rows=1702 width=13) (actual time=2.306..9.052 rows=1500.00
      loops=2)
                     Hash Cond: (co.id_cliente = c.id_cliente)
                     Buffers: shared hit=1904
                     ->  Parallel Seq Scan on comprobante co  (cost=0.00..3986.47 rows=1702 width=4) (actual
      time=0.007..6.176 rows=1500.00 loops=2)
                           Filter: ((id_sala = 29) AND (fecha >= (CURRENT_DATE - '7 days'::interval)) AND (fecha <
      (CURRENT_DATE + '1 day'::interval)))
                           Rows Removed by Filter: 73500
                           Buffers: shared hit=1560
                     ->  Hash  (cost=544.00..544.00 rows=20000 width=13) (actual time=4.535..4.536 rows=20000.00
      loops=1)
                           Buckets: 32768  Batches: 1  Memory Usage: 1194kB
                           Buffers: shared hit=344
                           ->  Seq Scan on cliente c  (cost=0.00..544.00 rows=20000 width=13) (actual
      time=0.012..2.560 rows=20000.00 loops=1)
                                 Buffers: shared hit=344
 Planning:
   Buffers: shared hit=168
 Planning Time: 3.708 ms
 Execution Time: 35.317 ms
(28 rows)
```

A2. S3 — después del índice

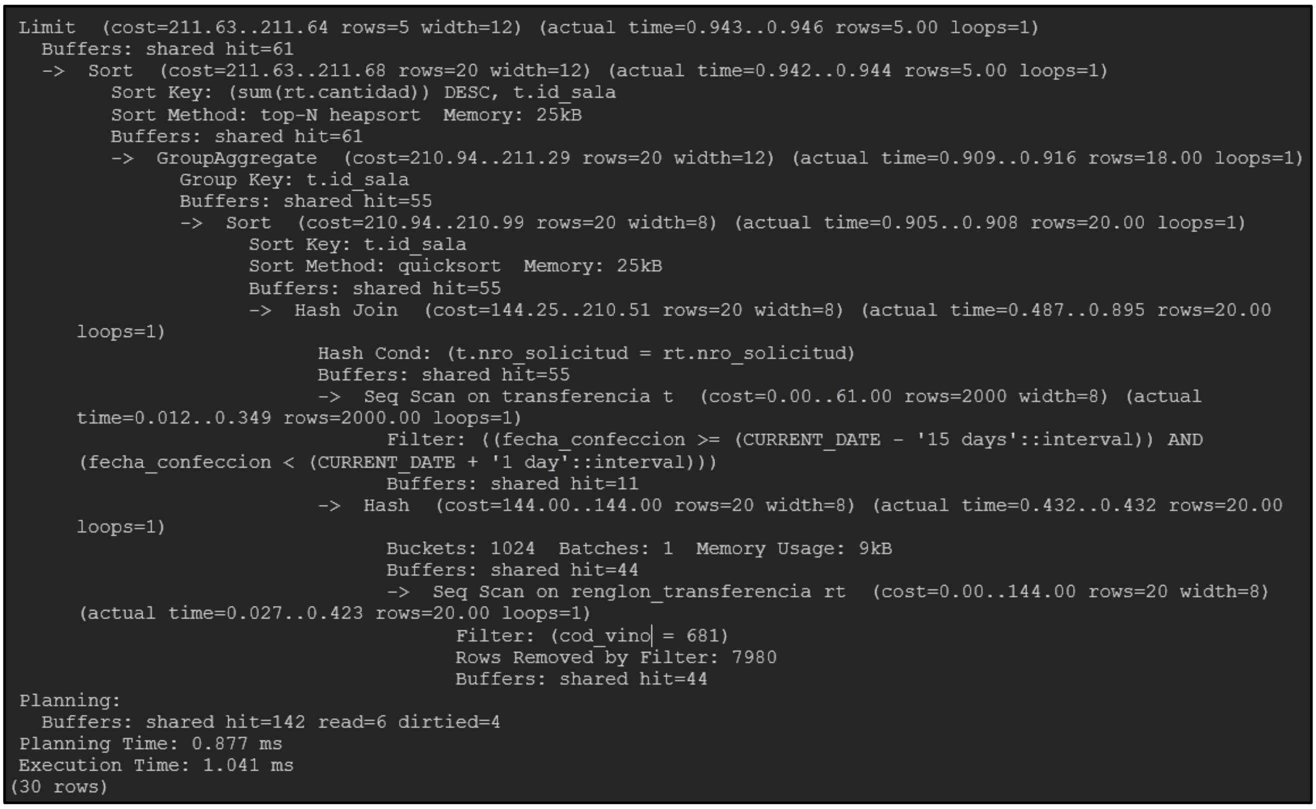
Lautaro · 12,623 ms · Archivo: s3_despues_1.txt



```
 GroupAggregate  (cost=2719.46..2741.19 rows=3 width=17) (actual time=12.173..12.398 rows=3.00 loops=1)
   Group Key: c.tipo_cliente
   Buffers: shared hit=1897 read=14
   ->  Sort  (cost=2719.46..2726.69 rows=2894 width=13) (actual time=12.044..12.138 rows=3000.00 loops=1)
         Sort Key: c.tipo_cliente, c.id_cliente
         Sort Method: quicksort  Memory: 198kB
         Buffers: shared hit=1897 read=14
         ->  Hash Join  (cost=879.33..2553.07 rows=2894 width=13) (actual time=5.417..10.340 rows=3000.00 loops=1)
               Hash Cond: (co.id_cliente = c.id_cliente)
               Buffers: shared hit=1891 read=14
               ->  Bitmap Heap Scan on comprobante co  (cost=85.33..1751.47 rows=2894 width=4) (actual
      time=0.610..4.856 rows=3000.00 loops=1)
                     Recheck Cond: ((id_sala = 29) AND (fecha >= (CURRENT_DATE - '7 days'::interval)) AND (fecha <
      (CURRENT_DATE + '1 day'::interval)))
                     Heap Blocks: exact=1547
                     Buffers: shared hit=1547 read=14
                     ->  Bitmap Index Scan on idx_comprobante_sala_fecha  (cost=0.00..84.61 rows=2894 width=0)
      (actual time=0.466..0.467 rows=3000.00 loops=1)
                           Index Cond: ((id_sala = 29) AND (fecha >= (CURRENT_DATE - '7 days'::interval)) AND (fecha
      < (CURRENT_DATE + '1 day'::interval)))
                           Index Searches: 1
                           Buffers: shared read=14
               ->  Hash  (cost=544.00..544.00 rows=20000 width=13) (actual time=4.679..4.679 rows=20000.00 loops=1)
                     Buckets: 32768  Batches: 1  Memory Usage: 1194kB
                     Buffers: shared hit=344
                     ->  Seq Scan on cliente c  (cost=0.00..544.00 rows=20000 width=13) (actual time=0.023..2.615
      rows=20000.00 loops=1)
                           Buffers: shared hit=344
 Planning:
   Buffers: shared hit=200 read=1
 Planning Time: 4.802 ms
 Execution Time: 12.623 ms
(27 rows)
```

A3. S4 — antes del índice

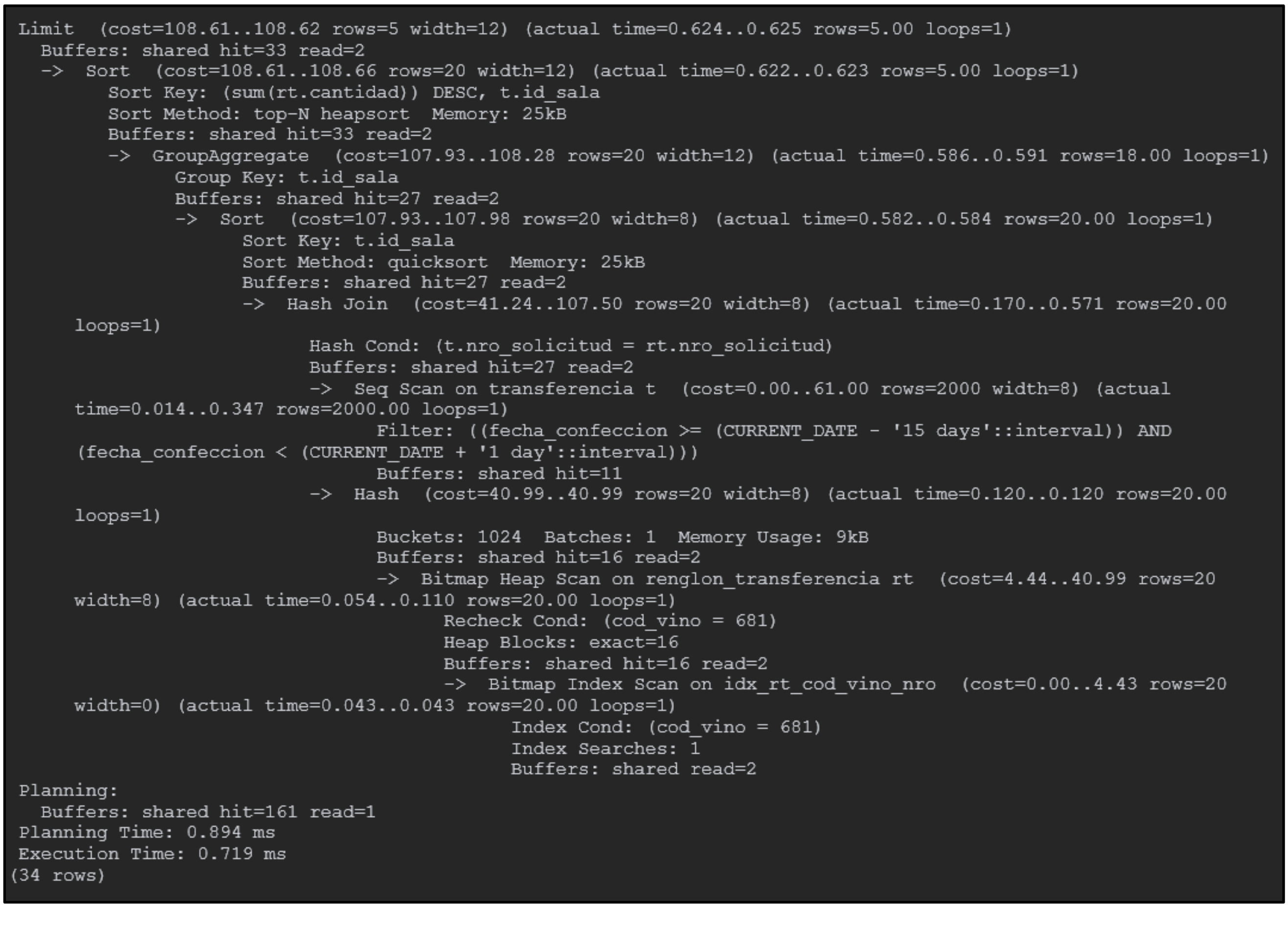
José · 1,041 ms · Archivo: s4_antes_1.txt



```
 Limit  (cost=211.63..211.64 rows=5 width=12) (actual time=0.943..0.946 rows=5.00 loops=1)
   Buffers: shared hit=61
   ->  Sort  (cost=211.63..211.68 rows=20 width=12) (actual time=0.942..0.944 rows=5.00 loops=1)
         Sort Key: (sum(rt.cantidad)) DESC, t.id_sala
         Sort Method: top-N heapsort  Memory: 25kB
         Buffers: shared hit=61
         ->  GroupAggregate  (cost=210.94..211.29 rows=20 width=12) (actual time=0.909..0.916 rows=18.00 loops=1)
               Group Key: t.id_sala
               Buffers: shared hit=55
               ->  Sort  (cost=210.94..210.99 rows=20 width=8) (actual time=0.905..0.908 rows=20.00 loops=1)
                     Sort Key: t.id_sala
                     Sort Method: quicksort  Memory: 25kB
                     Buffers: shared hit=55
                     ->  Hash Join  (cost=144.25..210.51 rows=20 width=8) (actual time=0.487..0.895 rows=20.00
      loops=1)
                           Hash Cond: (t.nro_solicitud = rt.nro_solicitud)
                           Buffers: shared hit=55
                           ->  Seq Scan on transferencia t  (cost=0.00..61.00 rows=2000 width=8) (actual
      time=0.012..0.349 rows=2000.00 loops=1)
                                 Filter: ((fecha_confeccion >= (CURRENT_DATE - '15 days'::interval)) AND
      (fecha_confeccion < (CURRENT_DATE + '1 day'::interval)))
                                 Buffers: shared hit=11
                           ->  Hash  (cost=144.00..144.00 rows=20 width=8) (actual time=0.432..0.432 rows=20.00
      loops=1)
                                 Buckets: 1024  Batches: 1  Memory Usage: 9kB
                                 Buffers: shared hit=44
                                 ->  Seq Scan on renglon_transferencia rt  (cost=0.00..144.00 rows=20 width=8)
      (actual time=0.027..0.423 rows=20.00 loops=1)
                                       Filter: (cod_vino = 681)
                                       Rows Removed by Filter: 7980
                                       Buffers: shared hit=44
 Planning:
   Buffers: shared hit=142 read=6 dirtied=4
 Planning Time: 0.877 ms
 Execution Time: 1.041 ms
(30 rows)
```

A4. S4 — después del índice

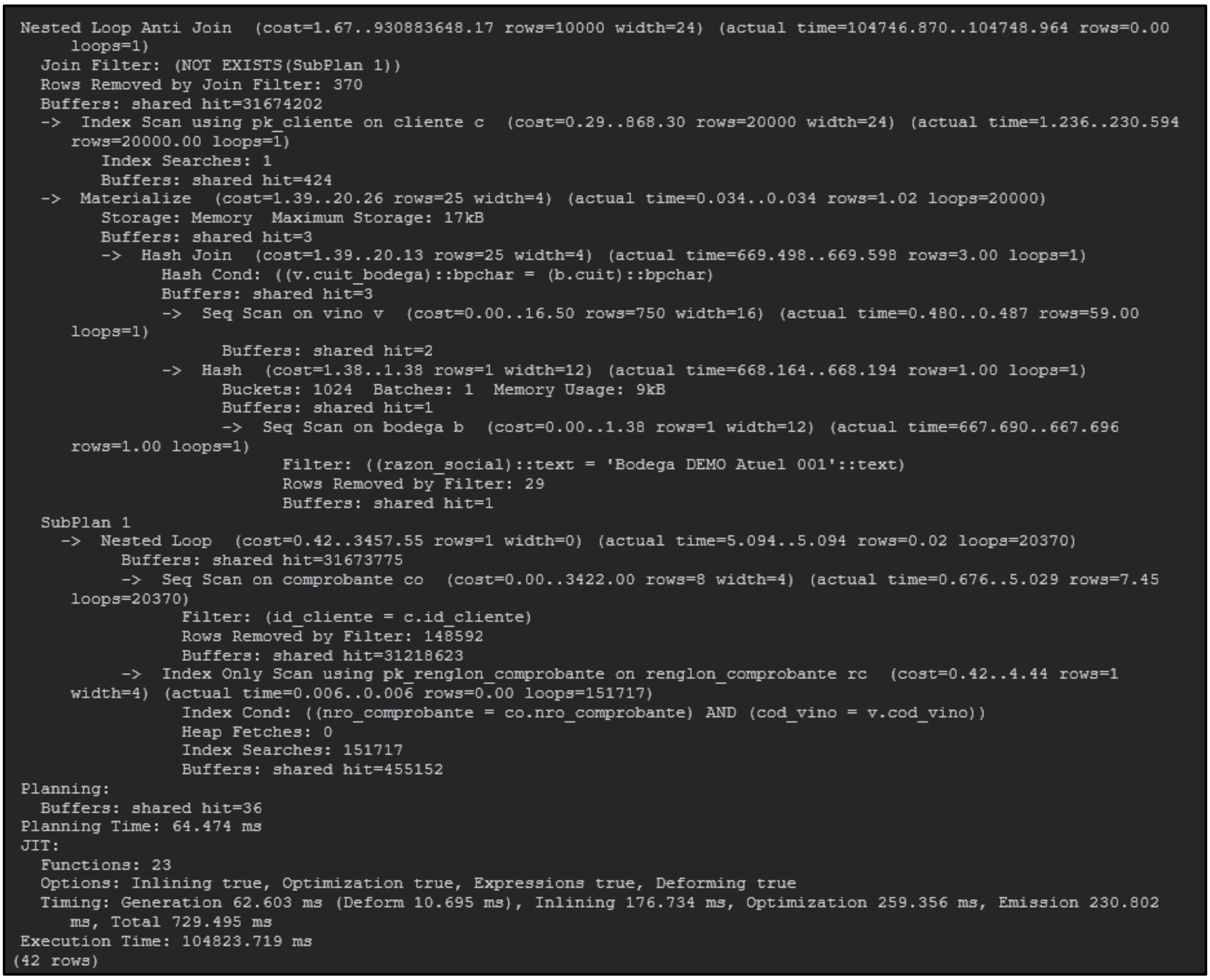
José · 0,719 ms · Archivo: s4_despues_1.txt



```
 Limit  (cost=108.61..108.62 rows=5 width=12) (actual time=0.624..0.625 rows=5.00 loops=1)
   Buffers: shared hit=33 read=2
   ->  Sort  (cost=108.61..108.66 rows=20 width=12) (actual time=0.622..0.623 rows=5.00 loops=1)
         Sort Key: (sum(rt.cantidad)) DESC, t.id_sala
         Sort Method: top-N heapsort  Memory: 25kB
         Buffers: shared hit=33 read=2
         ->  GroupAggregate  (cost=107.93..108.28 rows=20 width=12) (actual time=0.586..0.591 rows=18.00 loops=1)
               Group Key: t.id_sala
               Buffers: shared hit=27 read=2
               ->  Sort  (cost=107.93..107.98 rows=20 width=8) (actual time=0.582..0.584 rows=20.00 loops=1)
                     Sort Key: t.id_sala
                     Sort Method: quicksort  Memory: 25kB
                     Buffers: shared hit=27 read=2
                     ->  Hash Join  (cost=41.24..107.50 rows=20 width=8) (actual time=0.170..0.571 rows=20.00
      loops=1)
                           Hash Cond: (t.nro_solicitud = rt.nro_solicitud)
                           Buffers: shared hit=27 read=2
                           ->  Seq Scan on transferencia t  (cost=0.00..61.00 rows=2000 width=8) (actual
      time=0.014..0.347 rows=2000.00 loops=1)
                                 Filter: ((fecha_confeccion >= (CURRENT_DATE - '15 days'::interval)) AND
      (fecha_confeccion < (CURRENT_DATE + '1 day'::interval)))
                                 Buffers: shared hit=11
                           ->  Hash  (cost=40.99..40.99 rows=20 width=8) (actual time=0.120..0.120 rows=20.00
      loops=1)
                                 Buckets: 1024  Batches: 1  Memory Usage: 9kB
                                 Buffers: shared hit=16 read=2
                                 ->  Bitmap Heap Scan on renglon_transferencia rt  (cost=4.44..40.99 rows=20
      width=8) (actual time=0.054..0.110 rows=20.00 loops=1)
                                       Recheck Cond: (cod_vino = 681)
                                       Heap Blocks: exact=16
                                       Buffers: shared hit=16 read=2
                                       ->  Bitmap Index Scan on idx_rt_cod_vino_nro  (cost=0.00..4.43 rows=20
      width=0) (actual time=0.043..0.043 rows=20.00 loops=1)
                                             Index Cond: (cod_vino = 681)
                                             Index Searches: 1
                                             Buffers: shared read=2
 Planning:
   Buffers: shared hit=161 read=1
 Planning Time: 0.894 ms
 Execution Time: 0.719 ms
(34 rows)
```

A5. S12-A — antes del índice

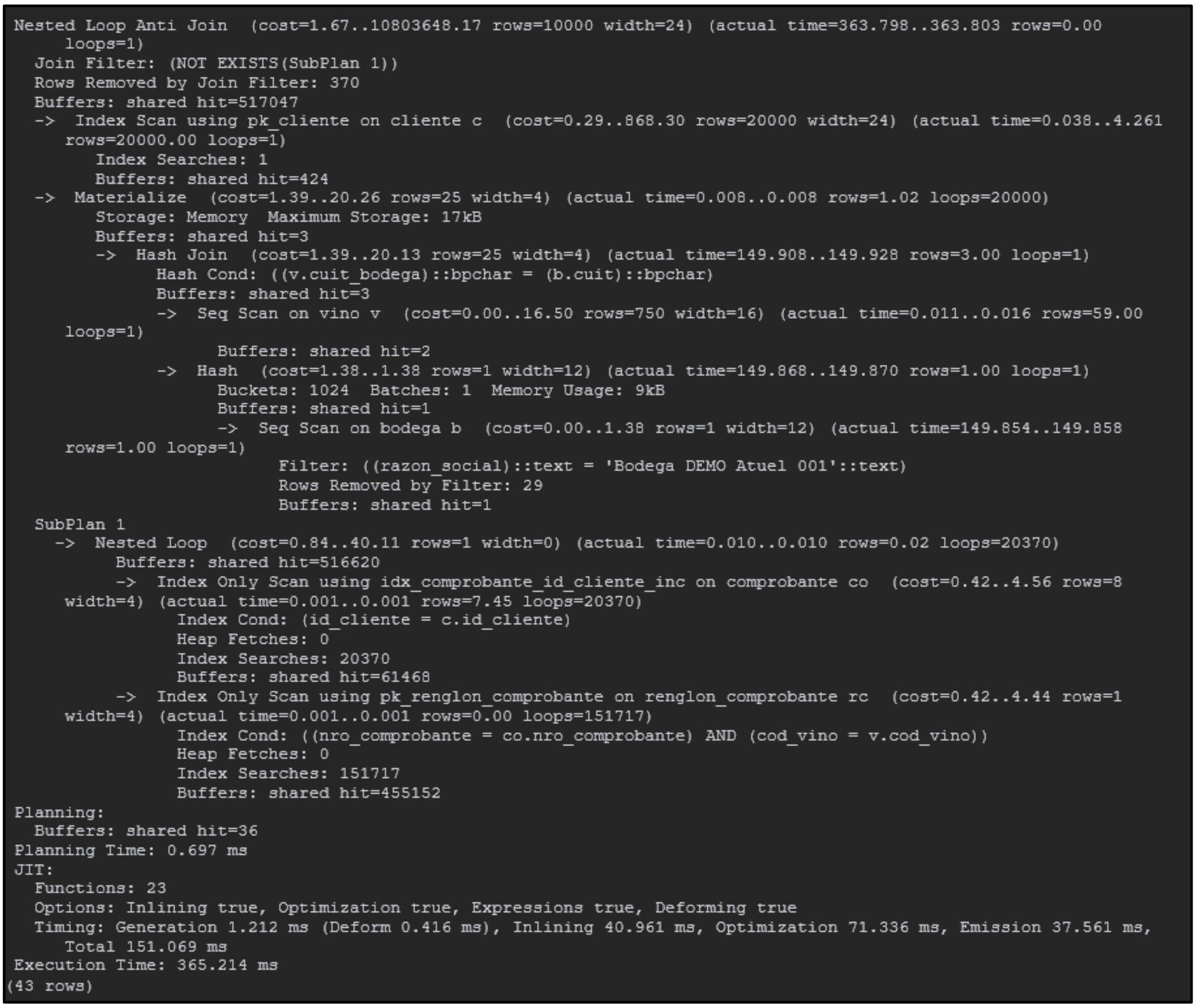
Antonio · 104.823,719 ms · Archivo: s12a_antes_2.txt



```
Nested Loop Anti Join  (cost=1.67..930883648.17 rows=10000 width=24) (actual time=104746.870..104748.964 rows=0.00
     loops=1)
  Join Filter: (NOT EXISTS(SubPlan 1))
  Rows Removed by Join Filter: 370
  Buffers: shared hit=31674202
  ->  Index Scan using pk_cliente on cliente c  (cost=0.29..868.30 rows=20000 width=24) (actual time=1.236..230.594
     rows=20000.00 loops=1)
        Index Searches: 1
        Buffers: shared hit=424
  ->  Materialize  (cost=1.39..20.26 rows=25 width=4) (actual time=0.034..0.034 rows=1.02 loops=20000)
        Storage: Memory  Maximum Storage: 17kB
        Buffers: shared hit=3
        ->  Hash Join  (cost=1.39..20.13 rows=25 width=4) (actual time=669.498..669.598 rows=3.00 loops=1)
              Hash Cond: ((v.cuit_bodega)::bpchar = (b.cuit)::bpchar)
              Buffers: shared hit=3
              ->  Seq Scan on vino v  (cost=0.00..16.50 rows=750 width=16) (actual time=0.480..0.487 rows=59.00
     loops=1)
                    Buffers: shared hit=2
              ->  Hash  (cost=1.38..1.38 rows=1 width=12) (actual time=668.164..668.194 rows=1.00 loops=1)
                    Buckets: 1024  Batches: 1  Memory Usage: 9kB
                    Buffers: shared hit=1
                    ->  Seq Scan on bodega b  (cost=0.00..1.38 rows=1 width=12) (actual time=667.690..667.696
     rows=1.00 loops=1)
                          Filter: ((razon_social)::text = 'Bodega DEMO Atuel 001'::text)
                          Rows Removed by Filter: 29
                          Buffers: shared hit=1
  SubPlan 1
    ->  Nested Loop  (cost=0.42..3457.55 rows=1 width=0) (actual time=5.094..5.094 rows=0.02 loops=20370)
          Buffers: shared hit=31673775
          ->  Seq Scan on comprobante co  (cost=0.00..3422.00 rows=8 width=4) (actual time=0.676..5.029 rows=7.45
     loops=20370)
                Filter: (id_cliente = c.id_cliente)
                Rows Removed by Filter: 148592
                Buffers: shared hit=31218623
          ->  Index Only Scan using pk_renglon_comprobante on renglon_comprobante rc  (cost=0.42..4.44 rows=1
     width=4) (actual time=0.006..0.006 rows=0.00 loops=151717)
                Index Cond: ((nro_comprobante = co.nro_comprobante) AND (cod_vino = v.cod_vino))
                Heap Fetches: 0
                Index Searches: 151717
                Buffers: shared hit=455152
Planning:
  Buffers: shared hit=36
Planning Time: 64.474 ms
JIT:
  Functions: 23
  Options: Inlining true, Optimization true, Expressions true, Deforming true
  Timing: Generation 62.603 ms (Deform 10.695 ms), Inlining 176.734 ms, Optimization 259.356 ms, Emission 230.802
     ms, Total 729.495 ms
Execution Time: 104823.719 ms
(42 rows)
```

A6. S12-A — después del INCLUDE

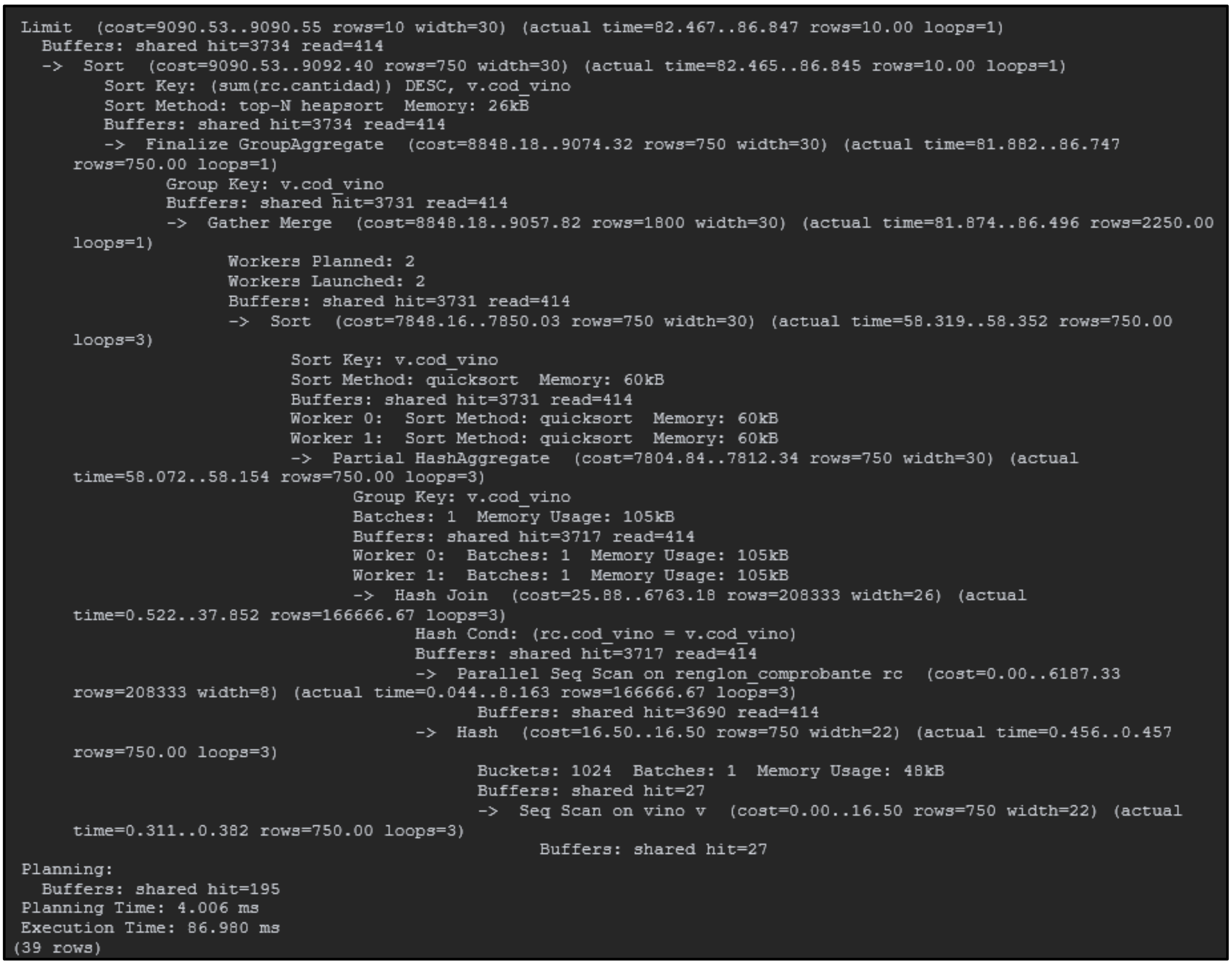
Antonio · 365,214 ms · Archivo: s12a_include_3.txt



```
 Nested Loop Anti Join  (cost=1.67..10803648.17 rows=10000 width=24) (actual time=363.798..363.803 rows=0.00
      loops=1)
   Join Filter: (NOT EXISTS(SubPlan 1))
   Rows Removed by Join Filter: 370
   Buffers: shared hit=517047
   ->  Index Scan using pk_cliente on cliente c  (cost=0.29..868.30 rows=20000 width=24) (actual time=0.038..4.261
      rows=20000.00 loops=1)
         Index Searches: 1
         Buffers: shared hit=424
   ->  Materialize  (cost=1.39..20.26 rows=25 width=4) (actual time=0.008..0.008 rows=1.02 loops=20000)
         Storage: Memory  Maximum Storage: 17kB
         Buffers: shared hit=3
         ->  Hash Join  (cost=1.39..20.13 rows=25 width=4) (actual time=149.908..149.928 rows=3.00 loops=1)
               Hash Cond: ((v.cuit_bodega)::bpchar = (b.cuit)::bpchar)
               Buffers: shared hit=3
               ->  Seq Scan on vino v  (cost=0.00..16.50 rows=750 width=16) (actual time=0.011..0.016 rows=59.00
      loops=1)
                     Buffers: shared hit=2
               ->  Hash  (cost=1.38..1.38 rows=1 width=12) (actual time=149.868..149.870 rows=1.00 loops=1)
                     Buckets: 1024  Batches: 1  Memory Usage: 9kB
                     Buffers: shared hit=1
                     ->  Seq Scan on bodega b  (cost=0.00..1.38 rows=1 width=12) (actual time=149.854..149.858
      rows=1.00 loops=1)
                           Filter: ((razon_social)::text = 'Bodega DEMO Atuel 001'::text)
                           Rows Removed by Filter: 29
                           Buffers: shared hit=1
   SubPlan 1
     ->  Nested Loop  (cost=0.84..40.11 rows=1 width=0) (actual time=0.010..0.010 rows=0.02 loops=20370)
           Buffers: shared hit=516620
           ->  Index Only Scan using idx_comprobante_id_cliente_inc on comprobante co  (cost=0.42..4.56 rows=8
      width=4) (actual time=0.001..0.001 rows=7.45 loops=20370)
                 Index Cond: (id_cliente = c.id_cliente)
                 Heap Fetches: 0
                 Index Searches: 20370
                 Buffers: shared hit=61468
           ->  Index Only Scan using pk_renglon_comprobante on renglon_comprobante rc  (cost=0.42..4.44 rows=1
      width=4) (actual time=0.001..0.001 rows=0.00 loops=151717)
                 Index Cond: ((nro_comprobante = co.nro_comprobante) AND (cod_vino = v.cod_vino))
                 Heap Fetches: 0
                 Index Searches: 151717
                 Buffers: shared hit=455152
 Planning:
   Buffers: shared hit=36
 Planning Time: 0.697 ms
 JIT:
   Functions: 23
   Options: Inlining true, Optimization true, Expressions true, Deforming true
   Timing: Generation 1.212 ms (Deform 0.416 ms), Inlining 40.961 ms, Optimization 71.336 ms, Emission 37.561 ms,
      Total 151.069 ms
 Execution Time: 365.214 ms
(43 rows)
```

A7. S1 — índice existente no utilizado

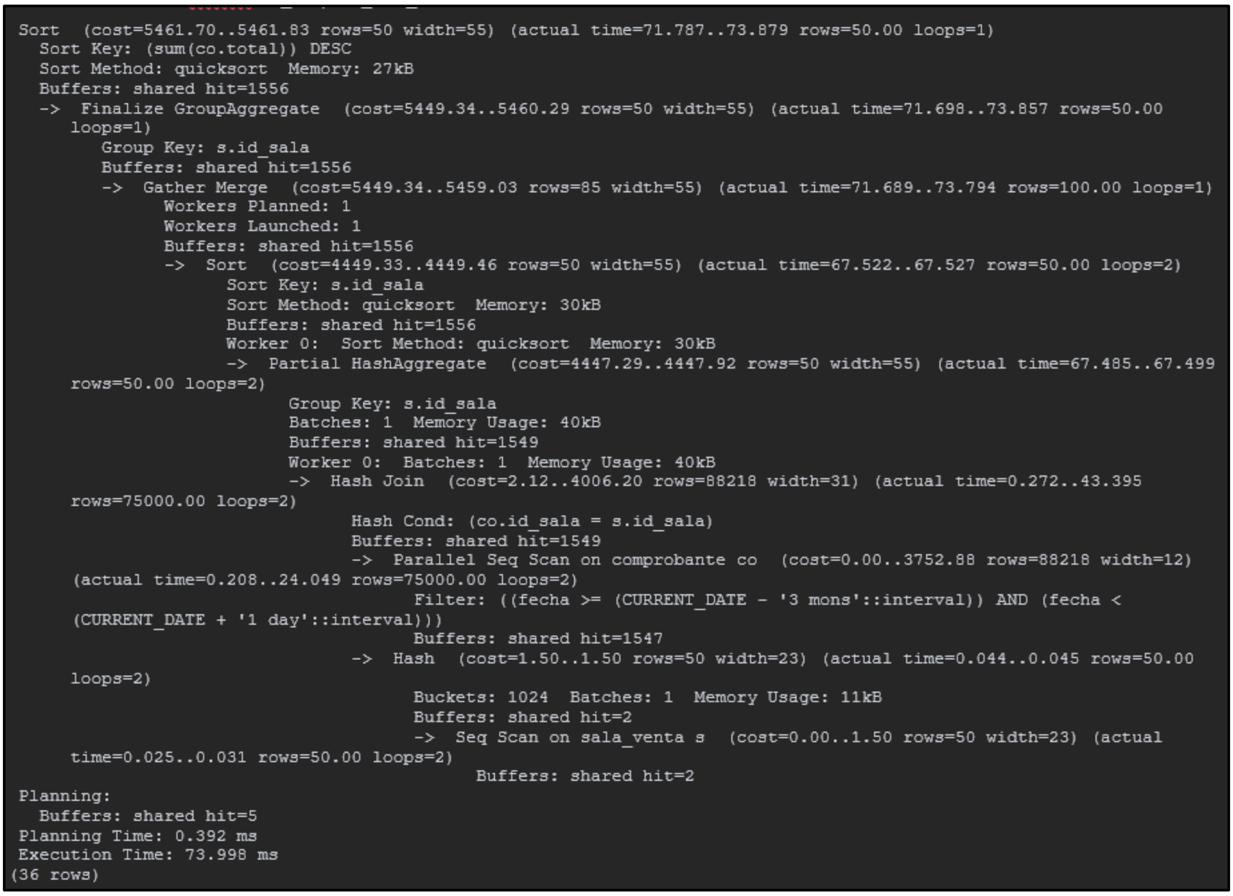
Lautaro · 86,980 ms · Archivo: s1_despues_2.txt



```
 Limit  (cost=9090.53..9090.55 rows=10 width=30) (actual time=82.467..86.847 rows=10.00 loops=1)
   Buffers: shared hit=3734 read=414
   ->  Sort  (cost=9090.53..9092.40 rows=750 width=30) (actual time=82.465..86.845 rows=10.00 loops=1)
         Sort Key: (sum(rc.cantidad)) DESC, v.cod_vino
         Sort Method: top-N heapsort  Memory: 26kB
         Buffers: shared hit=3734 read=414
         ->  Finalize GroupAggregate  (cost=8848.18..9074.32 rows=750 width=30) (actual time=81.882..86.747
      rows=750.00 loops=1)
               Group Key: v.cod_vino
               Buffers: shared hit=3731 read=414
               ->  Gather Merge  (cost=8848.18..9057.82 rows=1800 width=30) (actual time=81.874..86.496 rows=2250.00
      loops=1)
                     Workers Planned: 2
                     Workers Launched: 2
                     Buffers: shared hit=3731 read=414
                     ->  Sort  (cost=7848.16..7850.03 rows=750 width=30) (actual time=58.319..58.352 rows=750.00
      loops=3)
                           Sort Key: v.cod_vino
                           Sort Method: quicksort  Memory: 60kB
                           Buffers: shared hit=3731 read=414
                           Worker 0:  Sort Method: quicksort  Memory: 60kB
                           Worker 1:  Sort Method: quicksort  Memory: 60kB
                           ->  Partial HashAggregate  (cost=7804.84..7812.34 rows=750 width=30) (actual
      time=58.072..58.154 rows=750.00 loops=3)
                                 Group Key: v.cod_vino
                                 Batches: 1  Memory Usage: 105kB
                                 Buffers: shared hit=3717 read=414
                                 Worker 0:  Batches: 1  Memory Usage: 105kB
                                 Worker 1:  Batches: 1  Memory Usage: 105kB
                                 ->  Hash Join  (cost=25.88..6763.18 rows=208333 width=26) (actual
      time=0.522..37.852 rows=166666.67 loops=3)
                                       Hash Cond: (rc.cod_vino = v.cod_vino)
                                       Buffers: shared hit=3717 read=414
                                       ->  Parallel Seq Scan on renglon_comprobante rc  (cost=0.00..6187.33
      rows=208333 width=8) (actual time=0.044..8.163 rows=166666.67 loops=3)
                                             Buffers: shared hit=3690 read=414
                                       ->  Hash  (cost=16.50..16.50 rows=750 width=22) (actual time=0.456..0.457
      rows=750.00 loops=3)
                                             Buckets: 1024  Batches: 1  Memory Usage: 48kB
                                             Buffers: shared hit=27
                                             ->  Seq Scan on vino v  (cost=0.00..16.50 rows=750 width=22) (actual
      time=0.311..0.382 rows=750.00 loops=3)
                                                   Buffers: shared hit=27
 Planning:
   Buffers: shared hit=195
 Planning Time: 4.006 ms
 Execution Time: 86.980 ms
(39 rows)
```

A8. S8 — índice BRIN descartado

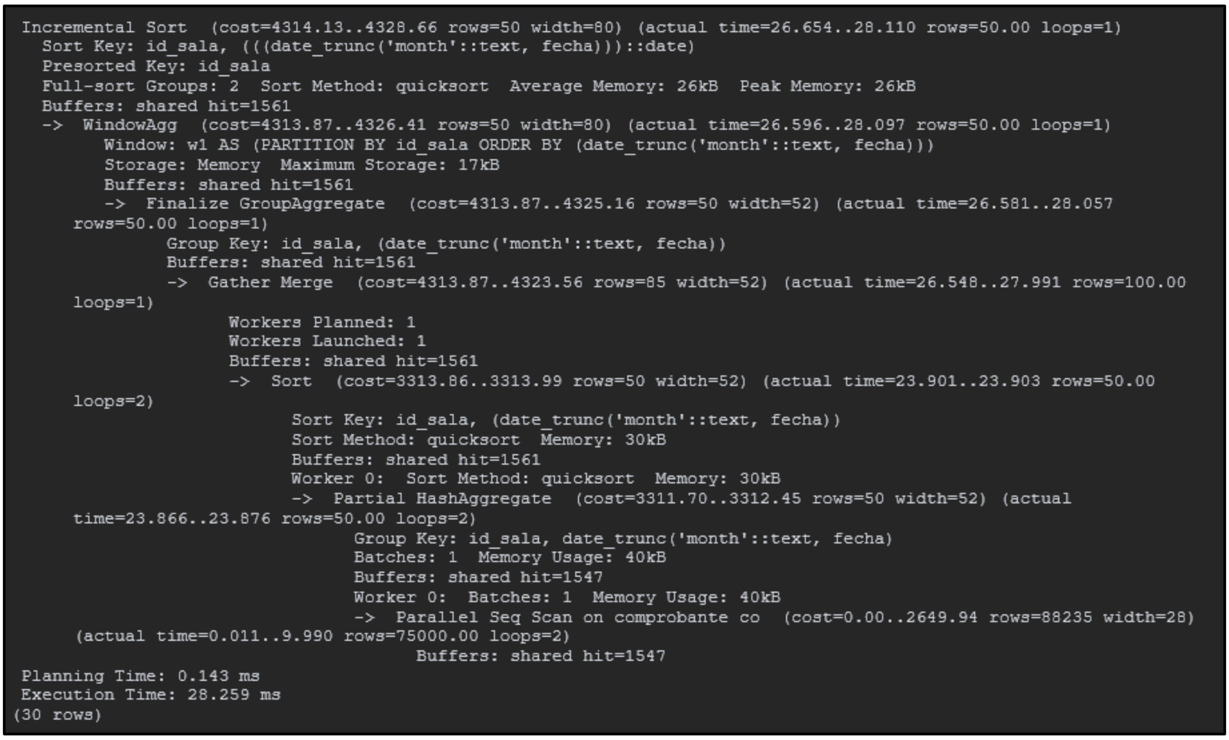
Antonio · 73,998 ms · Archivo: s8_despues_brin_2.txt



```
Sort  (cost=5461.70..5461.83 rows=50 width=55) (actual time=71.787..73.879 rows=50.00 loops=1)
  Sort Key: (sum(co.total)) DESC
  Sort Method: quicksort  Memory: 27kB
  Buffers: shared hit=1556
  ->  Finalize GroupAggregate  (cost=5449.34..5460.29 rows=50 width=55) (actual time=71.698..73.857 rows=50.00
     loops=1)
        Group Key: s.id_sala
        Buffers: shared hit=1556
        ->  Gather Merge  (cost=5449.34..5459.03 rows=85 width=55) (actual time=71.689..73.794 rows=100.00 loops=1)
              Workers Planned: 1
              Workers Launched: 1
              Buffers: shared hit=1556
              ->  Sort  (cost=4449.33..4449.46 rows=50 width=55) (actual time=67.522..67.527 rows=50.00 loops=2)
                    Sort Key: s.id_sala
                    Sort Method: quicksort  Memory: 30kB
                    Buffers: shared hit=1556
                    Worker 0:  Sort Method: quicksort  Memory: 30kB
                    ->  Partial HashAggregate  (cost=4447.29..4447.92 rows=50 width=55) (actual time=67.485..67.499
     rows=50.00 loops=2)
                          Group Key: s.id_sala
                          Batches: 1  Memory Usage: 40kB
                          Buffers: shared hit=1549
                          Worker 0:  Batches: 1  Memory Usage: 40kB
                          ->  Hash Join  (cost=2.12..4006.20 rows=88218 width=31) (actual time=0.272..43.395
     rows=75000.00 loops=2)
                                Hash Cond: (co.id_sala = s.id_sala)
                                Buffers: shared hit=1549
                                ->  Parallel Seq Scan on comprobante co  (cost=0.00..3752.88 rows=88218 width=12)
     (actual time=0.208..24.049 rows=75000.00 loops=2)
                                      Filter: ((fecha >= (CURRENT_DATE - '3 mons'::interval)) AND (fecha <
     (CURRENT_DATE + '1 day'::interval)))
                                      Buffers: shared hit=1547
                                ->  Hash  (cost=1.50..1.50 rows=50 width=23) (actual time=0.044..0.045 rows=50.00
     loops=2)
                                      Buckets: 1024  Batches: 1  Memory Usage: 11kB
                                      Buffers: shared hit=2
                                      ->  Seq Scan on sala_venta s  (cost=0.00..1.50 rows=50 width=23) (actual
     time=0.025..0.031 rows=50.00 loops=2)
                                            Buffers: shared hit=2
 Planning:
   Buffers: shared hit=5
 Planning Time: 0.392 ms
 Execution Time: 73.998 ms
(36 rows)
```

A9. S11-B — índice de expresión descartado

Antonio · 28,259 ms · Archivo: s11b_despues_expr_2.txt



```
 Incremental Sort  (cost=4314.13..4328.66 rows=50 width=80) (actual time=26.654..28.110 rows=50.00 loops=1)
   Sort Key: id_sala, (((date_trunc('month'::text, fecha)))::date)
   Presorted Key: id_sala
   Full-sort Groups: 2  Sort Method: quicksort  Average Memory: 26kB  Peak Memory: 26kB
   Buffers: shared hit=1561
   ->  WindowAgg  (cost=4313.87..4326.41 rows=50 width=80) (actual time=26.596..28.097 rows=50.00 loops=1)
         Window: w1 AS (PARTITION BY id_sala ORDER BY (date_trunc('month'::text, fecha)))
         Storage: Memory  Maximum Storage: 17kB
         Buffers: shared hit=1561
         ->  Finalize GroupAggregate  (cost=4313.87..4325.16 rows=50 width=52) (actual time=26.581..28.057
     rows=50.00 loops=1)
               Group Key: id_sala, (date_trunc('month'::text, fecha))
               Buffers: shared hit=1561
               ->  Gather Merge  (cost=4313.87..4323.56 rows=85 width=52) (actual time=26.548..27.991 rows=100.00
     loops=1)
                     Workers Planned: 1
                     Workers Launched: 1
                     Buffers: shared hit=1561
                     ->  Sort  (cost=3313.86..3313.99 rows=50 width=52) (actual time=23.901..23.903 rows=50.00
     loops=2)
                           Sort Key: id_sala, (date_trunc('month'::text, fecha))
                           Sort Method: quicksort  Memory: 30kB
                           Buffers: shared hit=1561
                           Worker 0:  Sort Method: quicksort  Memory: 30kB
                           ->  Partial HashAggregate  (cost=3311.70..3312.45 rows=50 width=52) (actual
     time=23.866..23.876 rows=50.00 loops=2)
                                 Group Key: id_sala, date_trunc('month'::text, fecha)
                                 Batches: 1  Memory Usage: 40kB
                                 Buffers: shared hit=1547
                                 Worker 0:  Batches: 1  Memory Usage: 40kB
                                 ->  Parallel Seq Scan on comprobante co  (cost=0.00..2649.94 rows=88235 width=28)
      (actual time=0.011..9.990 rows=75000.00 loops=2)
                                       Buffers: shared hit=1547
 Planning Time: 0.143 ms
 Execution Time: 28.259 ms
(30 rows)
```